Building Global Sample Graph for Effective Code Clone Detection

Chaofan Li[b,1], Tongyang Xu[a,1], Shihan Dou[c], Jingyuan Li[a], Ju Jia[a,*], Yueming Wu[b] and Deqing Zou[b]

[a]School of Cyber Science and Engineering, Southeast University, Nanjing, China
[b]School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan, China
[c]Fudan NLP Lab, Fudan University, Shanghai, China

ARTICLE INFO

Keywords:
Clone Detection
Node Embedding
Code Metrics
Global Sample Graph
Code Representation Learning

ABSTRACT

Code clone detection aims to identify similar code fragments and is important for software maintenance and evolution. Existing techniques for code clone detection have achieved encouraging results, but most of them analyze code samples individually and do not explicitly exploit the underlying connections among code samples. In this paper, we propose *Gitor* to capture such connections for clone detection. Specifically, given a code database, we first tokenize code samples to extract individual information (*i.e.,* keywords and code metrics). We then construct a global sample graph in which each node denotes either a code sample or a type of individual information. Based on this graph, we apply node embedding to obtain vector representations of code samples. Subsequently, we either compute cosine similarity to efficiently detect clone pairs or input these vectors into a neural network classifier to identify complex semantic clones. We conduct experiments on a widely used benchmark dataset and compare *Gitor* against popular clone detectors and deep learning-based methods, including the recently proposed baseline *CC2Vec*, while also considering LLM-based approaches. Experimental results show that *Gitor* achieves an F1 score of 0.99 and outperforms the compared deep learning-based detectors. For scalability, *Gitor* completes clone detection on one million code pairs, achieving a runtime reduction of over 95% compared with the baseline *ASTNN*. Furthermore, experiments on multiple code metrics show that most metric combinations surpass 0.90 precision at moderate thresholds, indicating that code metrics can effectively contribute to clone detection.

1. Introduction

Code clone, also known as duplicate code or similar code, refers to the existence of two or more identical or similar source code fragments. Numerous empirical studies (Baker, 1995; Kim et al., 2005; Roy and Cordy, 2008) have shown that code cloning widely exists in different open source or closed source code bases. For example, (Baker, 1995; Patenaude et al., 1999a) detected 22.3% of code clones in Linux system, Kamiya et al. found 29% code clones in JDK, and clone prevalence has also been observed in more recent software ecosystems (Li et al., 2026). Widespread code cloning has helped the development of software systems to a certain extent and can have positive benefits (Kapser and Godfrey, 2006; Rattan et al., 2013). However, many studies have pointed out that a large number of code clones can have a negative impact on software systems maintenance (Mondal et al., 2019; Hu et al., 2021), since it may introduce bugs or vulnerabilities. Therefore, the automatic detection of code clones has attracted wide attention in the field of software engineering.

According to the syntactic or semantic similarity of code clones, Bellon et al. classified code clones into four types (Roy and Cordy, 2007; Bellon et al., 2007): textual similarity (type 1), lexical similarity (type 2), syntactic similarity (type 3), and semantic similarity (type 4). From type 1 to type 4, the similarity of cloned codes gradually decreases and the difficulty of detection gradually increases. A number of code clone detection method has been proposed (Svajlenko et al., 2014; Kamiya et al., 2002; Sajnani et al., 2016; White et al., 2016; Zhang et al., 2019b; Jiang et al., 2007; Wang et al., 2017; Zakeri-Nasrabadi et al., 2023). For example, a state-of-the-art token-based method namely *SourcererCC* (Sajnani et al., 2016) is designed to capture the tokens' overlap similarity among different methods to detect Type-1 to Type-3 clones. In practice, token-based techniques are unable to handle Type-4 clones (*i.e.,* semantic clones) due to a lack of respect for program semantics. To mitigate the issue, researchers use program analysis to distill the semantics of code fragments into tree or graph representations (*e.g.*, abstract syntax tree and control flow graph) and apply tree or graph matching to quantify the similarity between different codes. Empirical studies (Komondoor and Horwitz, 2001; Wang et al., 2017; Alomari and Stephan, 2022) have shown that tree-based and graph-based code clone detectors can achieve better performance on semantic code clone analysis. However, due to the complexity of tree and graph structures, they are unable to scale to large programs. Given that large-scale clone detection is essential to daily software engineering activities such as code search (Keivanloo et al., 2011), mining library candidates (Ishihara et al., 2012), and license violation detection (German et al., 2009; Koschke, 2012), there is an increasing need for a scalable technique to detect code clones.

In this paper, we propose a novel code clone detection method leveraging global graph built across code samples. We find that almost all current code clone detection methods focus on extracting the features from source code directly while ignoring the potential underlying connections among

*Corresponding author
jiaju@seu.edu.cn (J. Jia)
[1]These authors contributed equally and are co-first authors.

different code samples. To achieve scalable and accurate code clone detection, we consider extracting these connections to build "bridges" between code samples (*i.e.,* global graph) and using them to detect code clones. Specifically, we mainly address two challenges in our paper.

- *How to build the global graph from source code and represent it properly to retain code details?*
- *How to utilize the global graph across different source code samples to efficiently and accurately detect code clones?*

To tackle the first challenge, we choose keywords along with code metrics as the individual information to represent the source code samples. In detail, since the programming language of our experimental dataset is Java, we leverage the reserved words of Java as keywords. Meanwhile, to better capture the code details, we also extract another kind of information (*i.e.,* code metrics) such as the number of statements and the flow control of code. Because the extraction of keywords and code metrics can be achieved by simple lexical analysis, we can complete scalable code clone analysis.

To address the second challenge, we use keywords and code metrics as the "bridge" to connect different code samples. Specifically, we build a global sample graph to explicitly represent the underlying connections between all code samples. Each node in the graph represents a code sample or a kind of individual information (*i.e.,* keywords or code metrics). Each edge indicates whether a code sample contains such individual information. After constructing the global graph, we perform a node embedding technique on it to convert all code samples into corresponding vector representations. Given the generated vectors, we use *cosine similarity* for fast detection of simple code clones and a *neural network-based approach* to identify more complex semantic clones.

We design and implement *Gitor*, a code clone detection system, and evaluate it on a widely used dataset, namely a widely used large-scale benchmark dataset (BigCloneBench). Our evaluation results show that *Gitor* is superior to ten widely used state-of-the-art clone detection systems including *SourcererCC* (Sajnani et al., 2016), *CCFinder* (Kamiya et al., 2002), *Nicad* (Roy and Cordy, 2008), *Deckard* (Jiang et al., 2007), *CCAligner* (Wang et al., 2018), *Oreo* (Saini et al., 2018), *LVMapper* (Wu et al., 2020a), *NIL* (Nakagawa et al., 2021), *Siamese* (Ragkhitwetsagul and Krinke, 2019), and *CloneWorks* (Svajlenko and Roy, 2017). Moreover, we also compare *Gitor* with seven representative deep learning-based baselines, including *RtvNN* (White et al., 2016), *CDLH* (Wei and Li, 2017), *TBCNN* (Mou et al., 2016), *ASTNN* (Zhang et al., 2019b), *SCDetector* (Wu et al., 2020b), *FCCA* (Hua et al., 2020), and a recently proposed baseline, *CC2Vec* (Dou et al., 2024). Additionally, to conduct a more comprehensive evaluation of *Gitor*, we further compare it against Large Language Models (LLMs) such as *GPT-3.5turbo* (Ouyang et al., 2022), and *GPT-4* (Achiam et al., 2023). The results show that *Gitor* achieves highly competitive performance, particularly in detecting semantic clones. Finally, we examine the scalability of *Gitor* on one million code pairs. *Gitor* also outperforms most conventional and deep learning-based methods in terms of runtime efficiency.

The main contributions of this paper are as follows.

- We propose a novel method to detect code clones by building a global sample graph using keywords and code metrics. The constructed global graph can capture the underlying connections between different source code samples.
- We design a code clone detection system namely *Gitor* and introduce two distinct methods for code clone detection, each serving a different purpose. The first method directly calculates the cosine similarity, designed for efficient detection of simple code clones. The second method utilizes a neural network-based approach to detect more complex code clones, such as semantic clones.
- We conduct extensive experiments to evaluate *Gitor*, comparing it with 10 representative code clone detection methods, 7 deep learning-based methods, and 2 Large Language Model-based methods. Experimental results demonstrate that *Gitor* outperforms state-of-the-art clone detectors, with F1 improvements ranging from 4% to 17% over advanced deep learning methods. For scalability, *Gitor* processes one million code pairs with over 95% less runtime than *ASTNN*, while maintaining high detection accuracy across different clone types.

Paper organization. The remainder of the paper is organized as follows. Section 2 presents the background and motivation. Section 3 introduces our system. Section 4 reports the experimental results. Section 5 shows the discussions. Section 6 describes the related work. Section 7 concludes the present paper.

This paper is an extended version of our previous work (Shan et al., 2023) published in the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE) 2023. We significantly enhanced the previous conference version in the following aspects: I) We replaced the side information used in the previous work with 27 code metrics, which capture the logic, complexity, and structural characteristics of the source code (Section 3.2). To evaluate the impact of different metrics on detection performance, we grouped the code metrics and conducted extensive experiments on various combinations. II) In addition to using cosine similarity for clone detection, we introduced a neural network-based classifier with fully connected layers and ReLU activation functions (Section 3.4). This neural network enhances the detection of complex clones by capturing deeper semantic relationships between code samples. III) We systematically analyzed the performance of code metrics under different parameters, such as cosine similarity

thresholds and vector dimensions (Section 4.2). Additionally, we explored the detection performance of various combinations of code metrics. IV) We conducted experiments to evaluate the combination of code metrics and keywords for graph construction. This approach allows the global graph to leverage two different types of the code information (Section 4.4). V) To provide a comprehensive evaluation, we added several state-of-the-art clone detection systems for comparison, including popular approaches, deep learning-based approaches, and Large Language Models (Section 4.5). VI) We incorporated experiments using deep learning-based approaches for code clone detection. Specifically, we evaluated the performance of neural networks with varying numbers of layers, showcasing the impact of network depth on detection accuracy (Section 4.5). VII) To assess the scalability of *Gitor*, we randomly selected one million code pairs from the benchmark dataset and conducted large-scale experiments (Section 4.6). This demonstrates the capability of our system to handle extensive datasets efficiently.

2. DEFINITION AND MOTIVATION

2.1. Definitions

The paper utilizes the well-accepted definitions of code clones and clone types as follows:

```java
public static int fib(int i){
    int f1=0, f2=1, c=0;
    if((i == 0) || (i == 1)) return i;
    for (int j =2; j<=i; j++){
        c=f1+f2; f1=f2; f2=c;
    }
    return c;
}
```

Listing 1: Original (Func #0)

```java
public static int fib(int i){
    int f1=0, f2=1, c=0;
    if((i == 0) || (i == 1)) return i;
    for (int j =2; j<=i; j++){
        c=f1+f2; f1=f2; f2=c;
    }
    return c;
}
```

Listing 2: Type-1 (Func #1)

```java
public static int fib(int num){
    int f1=0, f2=1, c=0;
    if((num == 0) || (num == 1)) return num;
    for (int j =2; j<=num; j++){
        c=f1+f2; f1=f2; f2=c;
    }
    return c;
}
```

Listing 3: Type-2 (Func #2)

```java
public static int calFib(int num){
    int fib1=0, fib2=1;
    int t=0;
    if((num == 1) || (num == 0)) return num;
    for (int k =2; k<=num; k++){
        t=fib1+fib2; fib1=fib2; fib2=t;
    }
    return t;
}
```

Listing 4: Type-3 (Func #3)

```java
public static long calFib(long number){
    long f1=0, f2=1, c=0;
    switch(number){
        case 0:
            return 0;
        case 1:
            return 1;
        default:
            break;
    }
    while(number>=2){
        c=f1+f2; f1=f2; f2=c;
        number--;
    }
    return c;
}
```

Listing 5: Type-4 (Func #4)

In our paper, we use the following widely used definitions Roy and Cordy (2007); Bellon et al. (2007) of code clone types.

- **Type-1 (textual similarity)**: Identical code fragments, except for minor differences in white-space, layout, or comments.
- **Type-2 (lexical similarity)**: Structurally identical code fragments, in addition to Type-1 clone differences, there might be some differences in identifier names and literal values.
- **Type-3 (syntactic similarity)**: Modified similar code fragments that differ at the statement level. Besides the Type-1 and Type-2 clone, the fragments might have statements added, modified and/or removed compared to each other.
- **Type-4 (semantic similarity)**: Dissimilar code fragments with the same functionality but implemented in a syntactically different way.

To elaborate on different types of clones, Listings 1 to 5 present examples from Type-1 to Type-4 clones. The original code computes the Fibonacci number given the input value. The Type-1 clone (Listing 2) is identical to the original code. The Type-2 clone (Listing 3) differs only in identifier names (e.g., `i` is replaced with `num`), while preserving the same control flow and logic. These two clone types are therefore relatively easy to detect. The Type-3 clone (Listing 4) is syntactically similar to the original code but differs at the statement level. In particular, the method name, parameter names, and local variable names are changed, and the code structure is slightly reorganized. Nevertheless, it still computes the Fibonacci number using a similar iterative strategy. Detecting Type-3 clones is harder than the previous two types. Finally, the Type-4 clone (Listing 5) computes the Fibonacci number using a structurally different implementation. Although it adopts a different control structure (e.g., `switch` and `while` statements) and changes data types, its underlying functionality remains equivalent. As a result, detecting Type-4 clones requires deeper semantic understanding of code fragments.

2.2. Motivation

To illustrate the key insight of our proposed method, we leverage Func #0 and its corresponding type 3 and type 4 clones (*i.e.,* Func #3 and Func #4) as our analysis targets. As shown in Listings 1, 4, and 5, these examples are all used to calculate the Fibonacci number of the given order. According to the definition of code clone, the clone pair Fib_0*.java* and Fib_3*.java* are classified as Type-3 clone (*i.e.,* syntactic similarity) since they differ at the statement level. The clone pair Fib_0*.java* and Fib_4*.java* are classified as a Type-4 clone (*i.e.,* semantic clone) because they have syntactically dissimilar code to implement the same functionality.

These examples point to a broader challenge for many widely used clone detectors, not a limitation specific to *SourcererCC*. The common limitation stems from how these detectors obtain clone evidence: they first represent each code fragment independently and then compare fragment pairs mainly based on lexical or syntactic similarity. Specifically, *SourcererCC* measures token overlap between two code fragments, *NiCad* detects clones by comparing normalized code fragments at the text-line level, and *Deckard* maps AST subtrees to characteristic vectors and groups similar vectors for clone detection. Although these methods adopt different representations and matching strategies, they do not explicitly model corpus-level relationships among code samples through shared information, nor do they propagate such global relationships when estimating code similarity. As a result, clone pairs with low lexical or syntactic similarity can remain difficult to identify. To illustrate this limitation more concretely, we use *SourcererCC* as a representative example because it is widely used and has an intuitive overlap-based similarity formulation.

2.2.1. SourcererCC

We next use *SourcererCC* as a representative example to illustrate how overlap-based pairwise comparison detects possible clone pairs. *SourcererCC* utilizes the overlap of two source code blocks to compute the similarity since it intuitively captures the notion of overlap among different code blocks. For example, given two code blocks C_1 and C_2, the overlap similarity $S(C_1, C_2)$ is calculated as the number of tokens shared by C_1 and C_2.

$$S(C_1, C_2) = |C_1 \cap C_2|. \tag{1}$$

Given the threshold θ and the maximum number of tokens $T = \max(|C_1|, |C_2|)$, a pair of code blocks is considered as a clone pair when the ratio of overlap similarity and T is greater than the threshold θ.

$$\frac{S(C_1, C_2)}{T} \geq \theta. \tag{2}$$

This mechanism enables *SourcererCC* to identify clones with high token overlap. However, this similarity measure does not consider higher-level semantic information.

2.2.2. Keywords Similarity

Besides the overlap of code blocks, we further consider using reserved keywords in the source code to measure similarity. Keywords capture fundamental control structures and semantic constraints of the code, making them more informative than general tokens. Specifically, we extract the reserved keywords from each function and compute their overlap similarity in the same way as *SourcererCC*. Compared to raw token overlap, this approach provides a more reliable and semantically meaningful assessment of code similarity. However, it still relies on pairwise comparison and lacks a global view of how functions are related.

2.2.3. Graph Constructing

To achieve a more accurate clone detection, we need to extract code information to represent the source code, preferably some kind of global information that can reflect the connections between source code samples rather than analyze the information from code samples individually. We therefore extract individual information from each code sample using keywords and code metrics. In detail, keywords capture important control structures and semantic cues of the source code. Meanwhile, to better capture code details, we also extract another kind of information (*i.e.,* code metrics), such as statement composition and control-flow characteristics. Because both keywords and code metrics can be efficiently derived from source code, they allow us to build a scalable global sample graph that connects all code samples.

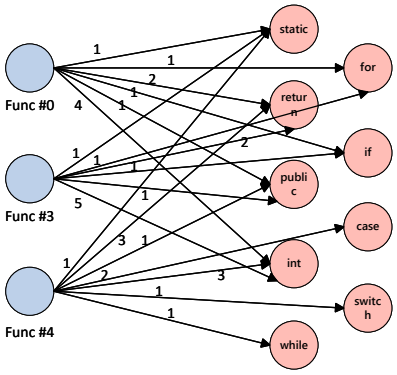


Figure 1: A global bipartite graph of Func #0, Func #3, and Func #4 by keywords.

We first tokenize the source code to get the sequences of tokens of Func #0, Func #3, and Func #4. Then we choose only the reserved words in Java as the keywords instead of all tokens to represent code samples since the reserved words are used in all Java source code samples. After extracting keywords from the above code samples, we construct a weighted directed graph with the frequency of keywords as weight as illustrated in Figure 1. This graph is a bipartite structure, where code samples and keywords form two disjoint sets of nodes. Each blue node represents a code sample, each red node represents a keyword, and the weight from blue nodes to red nodes is the frequency of keyword in the corresponding code sample. Subsequently, we calculate the values of selected code metrics for Func #0, Func #3, and Func #4. Using the same methodology, we

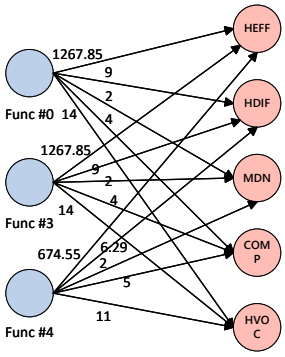


Figure 2: A global bipartite graph of Func #0, Func #3, and Func #4 by code metrics.

construct another bipartite graph, as illustrated in Figure 2. In this graph, the blue nodes still represent code samples, while the red nodes represent code metric types (*e.g.*, HEFF and HDIF, corresponding to Halstead effort and Halstead difficulty). The weight from a code sample node to a code metric node denotes the value of the corresponding metric for that sample.

Given n code samples, pairwise comparison requires $O(n^2)$ similarity checks, which becomes infeasible at scale. By contrast, the global graph is constructed once with linear complexity in the number of edges, making it much more scalable for large datasets.

2.2.4. Node Embedding

To assess the similarity among Func #0, Func #3, and Func #4 in Figures 1 and 2, we first apply node embedding methods to obtain their vector representations. Such methods are designed to capture structural information and inter-node relationships by transforming graph structures into representation vectors, thereby preserving underlying node similarity (Grover and Leskovec, 2016; Palumbo et al., 2018). In this paper, we use ProNE (Zhang et al., 2019a) to map code-sample nodes in the weighted global sample graph into vector representations, enabling the relationships induced by shared keywords and code metrics to be reflected in the embedding space.

2.2.5. Similarity Evaluation

We first calculate the similarity of above two code blocks using the method mentioned in *SourcererCC*. It shows that the number of tokens in Func #0 and Func #3 is 73 and 74, respectively. Then the same tokens shared by Func #0 and Func #3 are obtained for computing the overlap similarity. We observe that there are 18 same tokens shared by these two code blocks, which means the overlap similarity of Func #0 and Func #3 is 18/74=0.24. If similarity threshold in *SourcererCC* is set to 70%, which means that *SourcererCC* reports two methods as a clone pair only when the ratio of number of shared tokens and maximum number of tokens of them is larger than 70%. In this case, *SourcererCC* will cause a false negative by reporting Func #0 and Func #3 as a none clone pair. Also, the similarity between Func #0 and Func #4 according to *SourcererCC* is only 0.23, which also results in a false negative. This highlights a limitation of token-based approaches, as they fail to capture structural or semantic similarity when the same functionality is expressed using different tokens.

To improve upon this, we further compute similarity based on reserved keywords instead of all tokens. For each function, we extract the Java reserved keywords, count their frequencies, and calculate the overlap by dividing the number of shared keywords by the larger keyword count. For Func #0 and Func #3, they both contain 11 keywords, among which 10 are shared, yielding a similarity of 10/11=0.91. For Func #0 and Func #4, they both contain 14 keywords, with 4 in common, giving a similarity of 4/14=0.29. These results indicate that keywords provide a more semantically grounded measure than raw tokens, but the Type-4 pair (*i.e.,* Func #0 and Func #4) remains a false negative under 70% thresholds. This shows that although keywords provide stronger features than raw tokens, they still cannot reliably capture deeper semantic equivalence.

Finally, we perform node embeddings on the bipartite graphs shown in Figures 1 and 2, generating vector representations for Func #0, Func #3, and Func #4 to compute their similarities. Using the vectors obtained from the graph in Figure 1, we find that the similarity between Func #0 and Func #3 is 0.99, and the similarity between Func #0 and Func #4 is 0.65. Similarly, using the vectors obtained from the graph in Figure 2, the similarity between Func #0 and Func #3 remains 0.99, while the similarity between Func #0 and Func #4 increases to 0.88. These results demonstrate a significant improvement in similarity scores between the clone pairs. Unlike directly using keyword counts to compute overlap, the graph embedding captures latent structural relationships, such as indirect connections through shared features, which enhance the ability to detect clone pairs. Prior studies on bipartite graph embedding also show that such methods are effective in preserving meaningful distinctions even in dense and symmetric graph structures Giamphy et al. (2023); Yan et al. (2022). This is further supported by our motivation examples, where *SourcererCC* and keyword overlap failed to correctly detect the Type-4 pair, but the embedding approach succeeded. Overall, these findings show that the embedding approach can effectively identify clone pairs even in bipartite graphs and offers clear advantages over using raw keyword counts.

Based on the observation, we propose a novel code clone detection framework by considering the global relationships between different functions.

3. APPROACH

In this section, we introduce our proposed system, namely *Gitor*.

3.1. System Overview

As shown in Figure 3, *Gitor* consists of three main phases: *Global Graph Construction*, *Node Embedding*, and *Clone Detection*. Given a source code repository or database,

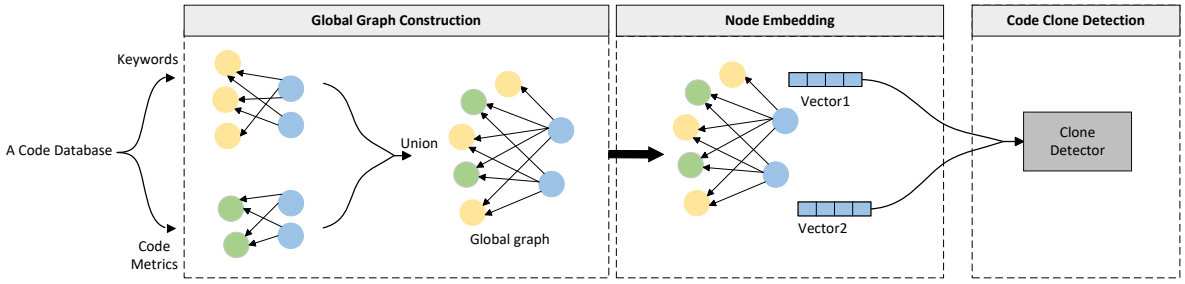


Figure 3: System overview of *Gitor*. Blue, yellow, and green nodes denote functions, keywords, and code metrics, respectively.

Gitor treats functions or methods as the basic analysis units and outputs the identified pairs of cloned functions.

Global Graph Construction: We first apply lexical analysis to extract the *individual information* including *keywords* and *code metrics* of a code sample with corresponding weights. Then a global sample graph is built by using these information where each node represents a sample or a type of *individual information*.

Node Embedding: Given a graph of the whole code base, we use a node embedding technique on the global graph and output the vectors of each node with chosen dimension. The input is a weighted global sample graph, and the output is a vector representation for each code sample in the code base.

Clone Detection: After the generation of vectors, we have two ways to detect potential clone pairs. First, we can simply calculate the cosine similarity of a pair of samples to identify code clones. Second, we can utilize a neural network as a classification model by inputting the vector representations of a pair of samples into the network to determine whether the code pair constitutes a clone.

3.2. Global Graph Construction

3.2.1. Individual Information Extraction

In this paper, we aim to combine the connection capture capability of graph embedding methods with the scalability of token-based methods. Therefore, we first conduct tokenization on the source code to extract the keywords and code metrics from the source code. Since our experiments are conducted on the Java-based benchmark dataset (BigCloneBench), we tokenize the *java* source code based on a java parse tool, namely *javalang* (javalang). We choose the *Java* reserved words as *keywords* along with 27 types of *code metrics* as *individual information.*

Table 1 presents 27 method-level metrics extracted from source code, some of which are derived from the open-source software quality observatory (SQO-OSS) (Samoladas et al., 2008). These metrics capture different aspects of code, including structural properties, complexity, and other relevant attributes. In addition to metrics from SQO-OSS, we also extract others that carry critical information. Researchers have observed (Saini et al., 2018) that for many clone pairs, capturing the types of literals used in methods is crucial, as this information can help distinguish methods that operate on different types and potentially implement

Table 1
Method-Level Software Metrics.

Name	Description	Name	Description
XMET	# external methods called	EXCT	# exceptions thrown
VREF	# variables referenced	EXCR	# exceptions referenced
VDEC	# variables declared	CREF	# classes referenced
NOS	# statements	COMP	Cyclomatic complexity
NOPR	# operators	CAST	# class casts
NOA	# arguments	NBLTRL*	# Boolean literals
NEXP	# expressions	NCLTRL*	# Character literals
NAND	# operands	NSLTRL*	# String literals
MDN	Max nesting depth	NNLTRL*	# Numerical literals
LOOP	# loops (for, while)	NNULLTRL*	# Null literals
LMET	# local methods called	NFCI**	# flow control (if, switch)
HVOC	Halstead vocabulary	MNP**	Max parallelism
HEFF	Halstead effort	NDI**	# numerical declarations
HDIF	Halstead difficulty		

different functionalities. Therefore, we include a set of metrics (*i.e.,* those marked with * in Table 1) that record the frequency of each type of literal used in a method. Finally, we select another group of metrics (*i.e.,* those marked with ** in Table 1) that provide deeper insights into code characteristics, offering a more comprehensive understanding of the code.

In this study, the selection of these metrics follows three guiding principles: (i) the metrics are either language-independent or can be adapted with minimal effort across programming languages; (ii) they capture complementary aspects of code beyond purely lexical information, such as structural patterns and complexity; and (iii) they have been shown in prior studies to be effective in distinguishing code fragments with similar syntax but different semantics. Consistent with the first principle, the implementation of these metrics can vary depending on the programming language used in the code samples. For instance, the functionality of curly braces in Python differs from that in Java. Therefore, our metric calculation functions are specifically designed to handle such syntactic variations accurately.

Additionally, we categorized the 27 code metrics into eight groups, as shown in Table 2, and systematically explored the classification performance of each group in clone detection through experimental evaluation. Since some metrics based on counts may correlate with code size, group-wise analysis allows us to better assess the contribution of

Table 2
Categorization of Code Metrics and Their Associated Metrics.

No.	Group Name	Included Code Metrics	Count
1	Halstead Metrics	HVOC, HEFF, HDIF	3
2	Literal Metrics	NBLTRL, NCLTRL, NSLTRL, NNLTRL, NNULLTRL	5
3	Exceptions and Class Reference Metrics	EXCT, EXCR, CREF, CAST	4
4	Code Structure Metrics	NOS, NOPR, NOA, NEXP, NAND, MDN, LOOP, COMP	8
5	Variables and Methods Metrics	VREF, VDEC, LMET, XMET	4
6	Complexity Metrics	COMP, HEFF, HDIF, HVOC, MDN	5
7	Global Call Metrics	XMET, CREF, EXCT	3
8	Local Call Metrics	LMET, VREF, MDN	3

different metric categories. The metric acronyms used in Table 2 are consistent with the definitions provided in Table 1. Halstead Metrics (*i.e.,* HVOC, HEFF, HDIF) includes Halstead complexity measures, which assess the cognitive effort and implementation difficulty. Literal Metrics (*i.e.,* NBLTRL, NCLTRL, NSLTRL, NNLTRL, NNULLTRL) capture different types of literals in the code. Exceptions and Class References Metrics (*i.e.,* EXCT, EXCR, CREF, CAST) focus on exception handling and dependencies. Code Structure Metrics (*i.e.,* NOS, NOPR, NOA, NEXP, NAND, MDN, LOOP, COMP) describe structural complexity, including basic structural properties. Variables and Methods Metrics (*i.e.,* VREF, VDEC, LMET, XMET) measure variable usage and method calls, reflecting modularity. Complexity Metrics (*i.e.,* COMP, HEFF, HDIF, HVOC, MDN) consolidate key complexity indicators. Global Call Metrics (*i.e.,* XMET, CREF, EXCT) evaluate external dependencies, while Local Call Metrics (*i.e.,* LMET, VREF, MDN) analyze internal interactions. This grouping ensures a structured analysis of different software complexity aspects, allowing for a targeted evaluation of code clone detection.

In our study on code clone detection, we identified the significance of both *keywords* and *code metrics*. Keywords, directly extracted from source code, serve as fundamental markers of code content. In contrast, *code metrics* delve deeper, capturing structural and complexity-related characteristics such as bracket depth, loop counts, and function complexity. To leverage the strengths of these two types of information, we constructed separate graph structures for keywords and code metrics. These graphs were then merged into a unified global graph to establish multidimensional connections between code samples. This approach not only integrates the lexical content and structural features of the code but also provides a more robust framework. By uncovering the intricate relationships and similarities between code fragments, it significantly enhances the accuracy of code clone detection.

After extracting *keywords* and *code metrics*, we can get a sequence of tokens with corresponding weight, which is the frequency in this case.

3.2.2. Global Graph Construction

Nowadays, the graph is an important kind of representation to encode relation structure, which is used in many domains (*i.e.,* social networks, citation networks, function call diagrams, etc.). The nodes and edges can represent the objects and relationships respectively. Evaluation of similarity between two nodes based on the graph structure has a wide range of applications, such as social networks analysis, knn, graph clustering, etc. Therefore, instead of simply comparing the similarity using the overlap *keywords* and *code metrics* of two samples, we first use *keywords* extracted from the last step to build a bipartite graph where functions and keywords are two disjoint sets of nodes. The edges connect function nodes to keyword nodes, and their weights indicate the frequency of keywords appearing in each function. To better illustrate this phase in *Gitor*, we

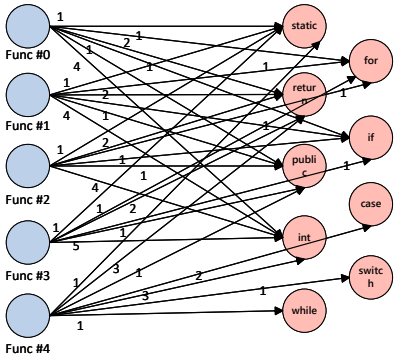


Figure 4: A global bipartite graph of Func #0-4 constructed by keywords.

choose the samples in Listing 1 as an example and present a clearer description in Figure 4, where the blue nodes represent Function #0-4 and the red nodes represent the keywords. The weight on the directed edges from functions to keywords is the frequency of keywords appearing in the functions. To better capture the program details of source code, we also select another kind of information. Specifically, we construct another graph using *code metrics* defined above, where the blue nodes represent functions, and the red nodes represent *code metric* types. Due to the large number of metrics (*i.e.,* 27 in total), we take complexity metrics as an example to illustrate our approach. In this case, the red nodes represent five complexity metrics (*e.g.*, COMP, HEFF, HDIF, HVOC, MDN), and the weight of edges represents the value of corresponding *code metrics*, as illustrated in Figure 5. After obtaining two graphs using *keywords* and *code metrics*, we merge them by unifying the nodes with the same labels to

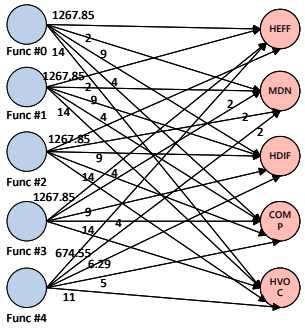


Figure 5: A global bipartite graph of Func #0-4 constructed by code metrics.

build a larger global sample graph. In this union graph, the edges from the keyword graph and the metric graph are preserved with their original weights. This design maintains the distinct semantics of the two information sources and allows their contributions to be automatically captured by the subsequent embedding process.

In short, the input of graph construction is a code database containing many code samples (*i.e.,* functions) and the output is a large global sample graph.

3.3. Node Embedding

Graph embedding aims to transform nodes in a graph into low-dimensional vector representations while preserving structural and semantic relationships among nodes (Dai et al., 2019). Such representations have been widely used in downstream tasks such as node classification and link prediction (Tang et al., 2015; Sun et al., 2019). Furthermore, recent advancements in graph neural networks have demonstrated robust capabilities in capturing environment-adaptive interactive relationships across diverse domains (Jia et al., 2025). Inspired by this expressiveness, after constructing the global sample graph, we apply graph embedding to obtain a vector representation for each code-sample node. The node embedding method adopted by *Gitor* should align with the properties of the proposed global sample graph. First, it should directly handle weighted graphs, since edge weights in our graph represent keyword frequencies and code metric values. Second, it should preserve both direct sample-feature connections and higher-order relationships propagated through the graph. Third, it should be lightweight and unsupervised, allowing the evaluation to focus on the proposed graph construction rather than task-specific training of the embedding model. ProNE meets these requirements by providing an efficient unsupervised embedding method for weighted graphs. Its spectral initialization captures local graph structures, while spectral propagation further incorporates higher-order proximity. This property is consistent with the design of *Gitor*, which aims to place code samples close in the embedding space not only when they share direct keywords or metrics, but also when they are connected through similar global graph contexts.

We also considered other categories of graph embedding methods. Random-walk methods, such as node2vec (Grover and Leskovec, 2016), can incur additional sampling overhead on large graphs. Neural aggregation methods, including GraphSAGE (Hamilton et al., 2017), typically require node features, neighborhood sampling, model training, and additional hyperparameter tuning. More recent scalable methods, such as Force2Vec (Rahman et al., 2022), are promising for general graph representation learning. However, using them would introduce an additional algorithmic variable and require separate validation in the context of our weighted global sample graph. Since the main objective of this work is to evaluate the effectiveness of the proposed global graph construction, rather than to benchmark graph embedding algorithms, we keep the node embedding method fixed and adopt ProNE as a practical choice.

Briefly, ProNE operates in two main steps. First, it derives an initial embedding by approximating a spectral representation of the weighted adjacency matrix, capturing local structural information such as the frequency of a keyword in a function. Second, it applies spectral propagation to incorporate higher-order proximity information, which is particularly important for revealing implicit relationships in our bipartite graph. Taking Figure 1 as an example, although *Func #0* and *Func #3* are not directly connected, they share common neighbor nodes (e.g., the keywords "*for*" and "*return*"). Through spectral propagation, structural information is propagated through these shared neighbors, resulting in more similar vector representations for *Func #0* and *Func #3* in the latent space. Moreover, the choice of embedding dimension d influences the trade-off between representational capacity and efficiency. Higher embedding dimensions provide greater expressiveness but increase computation and may yield diminishing returns. Therefore, we evaluate multiple embedding dimensions (*i.e.,* d = 16, 32, 64, 128) to identify the optimal embedding dimension for clone detection.

In brief, the input of node embedding is the graph constructed before, and the outputs are vectors of all nodes in the graph with the pre-defined dimension.

3.4. Clone Detection

After collecting the vectors of all functions, we propose two methods for code clone detection. The first method is to apply code clone detection by directly computing the cosine similarity between these vectors, which is particularly effective for detecting simple clones. Moreover, the second method employs neural networks as classifiers to determine whether two pieces of source code represented by these vectors are clone pairs. This approach is designed to handle complex or semantic clones. Figure 6 describes an example of these two detection methods of *Gitor*.

3.4.1. Method 1: Detect clones with cosine similarity

Cosine similarity is a commonly used metric, which measures similarity between two vectors, especially in high-dimension space. It measures similarity as the cosine of the angle between two vectors. Two similar vectors are expected to have a small angle between them. The cosine similarity of two vectors x and y is defined as follows:

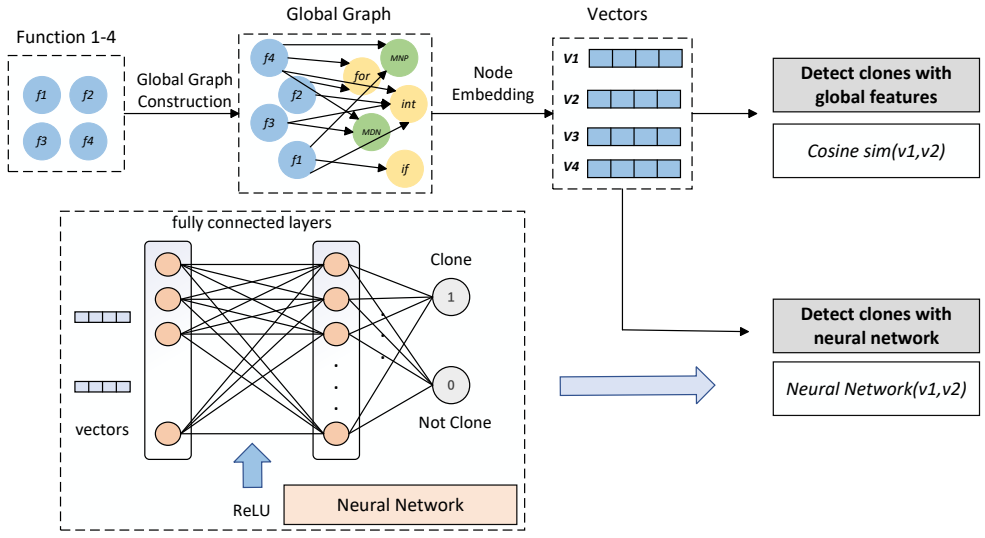


Figure 6: Two methods of *Gitor*. Blue, yellow, and green nodes denote functions, keywords, and code metrics, respectively. The first method uses cosine similarity for clone detection, and the second uses neural networks for clone detection.

$$\cos\theta = \frac{\sum_{i=1}^{d} x_i y_i}{\sqrt{\sum_{i=1}^{d} x_i^2}\sqrt{\sum_{i=1}^{d} y_i^2}} \tag{3}$$

Our first method is simply calculating the cosine similarity between two vectors, If the similarity is greater than a certain threshold (*e.g.*, 0.7), they are identified as a clone pair, as illustrated in Figure 6.

3.4.2. Method 2: Detect clones with neural networks

We choose the graph as the representation of the whole code base, as its natural structure can capture the underlying global connections among code samples more effectively than analyzing them individually. After obtaining the vector representations, we further introduce a lightweight neural classifier to better capture non-linear relationships between code pairs. While cosine similarity directly measures distance in the embedding space, it can be insufficient when clone pairs exhibit subtle structural or semantic similarities (*e.g.*, certain Type-3 or Type-4 clones). The neural network-based method addresses this limitation by learning non-linear decision boundaries over pairs of embeddings. Specifically, for a pair of code fragments with embedding vectors v_i and v_j, we construct the input feature x by concatenating them: $x = [v_i \oplus v_j] \in \mathbb{R}^{2d}$. This joint vector x is then fed into the neural network, where the output 0 indicates 'Not Clone' and 1 indicates 'Clone', as illustrated in Figure 6.

To explore the impact of network depth on detection performance, we conducted three experiments with different numbers of fully connected layers (*i.e.,* 1, 2, and 3 layers). *Gitor-1L* directly maps the concatenated vector to the two output logits without any hidden layer. *Gitor-2L* introduces one hidden layer with ReLU activation before the output layer. *Gitor-3L* further extends the network with two hidden layers, each followed by ReLU activation, before producing the final logits. In all experiments, the output layer provides the probability of a pair belonging to either the clone or the non-clone class through a softmax function. Training is performed with cross-entropy loss, optimized by Adam with a learning rate of 1×10^{-4}, batch size of 64, for 10 epochs. To prevent data leakage, we construct training and test sets in a function-disjoint manner, ensuring that no code fragment appears in both sets. Specifically, we first divide the functions in BigCloneBench into disjoint training and testing subsets. Within each subset, we then perform stratified sampling over clone types to construct clone pairs, and further sample an equal number of non-clone pairs to obtain balanced datasets. In this way, we construct a labeled dataset containing 100,000 pairs for training and 20,000 pairs for testing.

4. EXPERIMENTS

In this section, we conduct a series of comprehensive experiments to evaluate the performance of *Gitor* in code clone detection. We not only evaluate the performance of code metrics in detecting code clones but also the enhancement in detection accuracy achieved by combining code metrics with keywords. Additionally, we compare *Gitor* with other state-of-the-art code clone detectors to comprehensively validate the superiority of our method in code clone detection. To assess scalability, we conduct a detailed experiment to demonstrate the runtime overhead of *Gitor*, providing deeper insights into its computational efficiency. The experimental evaluation is structured around the design of the *individual information* used to construct the global sample graph. In particular, the first three research questions investigate the effectiveness of *keywords*, *code metrics*, and their combination for clone detection. To keep the comparison focused on representation design, other variables, such as the graph

embedding method, remain fixed. Specifically, we aim to address the following research questions:

- *RQ1: How effective is Gitor in detecting code clones when using keywords only?*
- *RQ2: What is the performance of Gitor in code clone detection using code metrics?*
- *RQ3: Can the combination of code metrics and keywords improve Gitor's detection performance?*
- *RQ4: How does Gitor perform in comparison to state-of-the-art code clone detection systems?*
- *RQ5: What is the runtime performance of Gitor in comparison with other state-of-the-art clone detectors?*

4.1. Experimental Settings

4.1.1. Dataset

We conduct our evaluations on the BigCloneBench (BigCloneBench), which consists of more than 8,000,000 labeled clone pairs from 25,000 systems. We use BigCloneBench as the primary benchmark because it is one of the most widely used datasets for function-level code clone detection and enables direct comparison with a broad range of prior studies. The code granularity of clone pairs in BigCloneBench is function-level, and each clone pair is manually assigned a corresponding clone type. Type-3 and Type-4 types are usually further divided into four subcategories based on their syntactical similarity score, as follows: i) *Very Strongly Type-3* (VST3) with a similarity between [0.9, 1.0), ii) *Strongly Type-3* (ST3) with a similarity between [0.7, 0.9), iii) *Moderately Type-3* (MT3) with a similarity between [0.5, 0.7), and iv) *Weakly Type-3/Type-4* (WT3/T4) with a similarity between [0.0, 0.5). The total number of these clone pairs used in our experiments is 8,446,574 including 8,139 Type-1 clones, 3,292 Type-2 clones, 4,577 VST3 clones, 3,469 ST3 clones, 7,606 MT3 clones, and 8,424,068 WT3/T4 clones. In the following experiment results, we use Type-4 (T4) to denote WT3/T4.

In addition to BigCloneBench, we use GoogleCodeJam as an additional benchmark for complementary validation. GoogleCodeJam is derived from an online programming competition held by Google and is commonly used for semantic clone detection evaluation (Zhao and Huang, 2018). It contains 1,669 projects from 12 competition problems. Following prior work, projects solving the same problem are treated as semantic clone pairs, while projects from different problems are treated as non-clone pairs, resulting in 275,570 semantic clone pairs and 1,116,376 non-clone pairs. Its different data source and problem distribution make it suitable for examining the applicability of *Gitor* beyond BigCloneBench.

In our graph-based experiment, each distinct function in the dataset is treated as a code sample node. After preprocessing, we obtained 73,319 function-level code sample nodes from the dataset. In addition, the graph contains 42 reserved keyword nodes and 27 code metric nodes. We employ all predefined Java reserved words as candidate keywords; consequently, the value 42 represents the number of distinct reserved words that actually appear in the preprocessed benchmark dataset. Specifically, the combined graph contains 73,388 nodes and 1,983,381 edges, reflecting the large scale and complexity of our constructed bipartite graph.

4.1.2. Implementation

For *individual information* extraction, we leverage a Java parser (*i.e., javalang* (javalang)) to extract the *keywords* and *code metrics* from source code samples. For global sample graph construction, we use a Python library, *networkx* (NetworkX), to build a weighted and directed graph. For node embedding, we employ a widely used embedding method, ProNE (Zhang et al., 2019a), to conduct the embedding of the graph. The output of embedding is a series of vectors of all nodes in the graph. We fix the graph embedding method to ProNE throughout RQ1–RQ3 to focus evaluations on representation design differences, avoiding an additional source of variation.

For similarity calculation, we apply *PyTorch* (PyTorch)'s cosine similarity function to compute the cosine similarity between these vectors, outputting a similarity score for the code pair. For the neural classifier described in Section 3.4.2, we implement and train the models in *PyTorch*, using the Adam optimizer (Diederik, 2014) with a learning rate of 0.0001 and cross-entropy loss (Paszke et al., 2019). The training batch size is 64.

All experiments are conducted on a server with Intel Xeon E5-2678 v3 @ 2.50GHz, 32 Gig-bytes memory, GeForce RTX 2080 TI Graphics Card and Ubuntu 18.04.5 LTS.

4.1.3. Comparative Tools

We also compare our approach with several state-of-the-art code clone detection systems. The selection of baselines is based on criteria of popularity, extensive use, and high citation counts in code clone detection research, as well as frequent inclusion in comparisons across various studies.

Specifically, we evaluate our method against twelve widely used code clone detectors: *SourcererCC* (Sajnani et al., 2016): An advanced clone detector that computes overlapping token similarities between methods for efficient and accurate clone identification. *CCFinder* (Kamiya et al., 2002): A token-based clone detection tool designed for large-scale codebases, which compares token sequences to identify potential clones. *Nicad* (Roy and Cordy, 2008): A detector that leverages the TXL parser to analyze code structure and identify syntactic clones with high precision. *Deckard* (Jiang et al., 2007): A tree-based clone detection tool that calculates Euclidean distances between feature vectors derived from abstract syntax trees (AST). *CCAligner* (Wang et al., 2018): A tool employing code window extraction and editing distance metrics to detect

code clones effectively. *Oreo* (Saini et al., 2018): A metric-based clone detector that combines information retrieval techniques with machine learning for improved accuracy. *LVMapper* (Wu et al., 2020a): A detector that identifies clones by computing shared token counts and using dynamic thresholding to balance precision and recall. *NIL* (Nakagawa et al., 2021): A method-level clone detector that relies on calculating the longest common subsequence (LCS) of token sequences to find clones. *Siamese* (Ragkhitwetsagul and Krinke, 2019): A token sequence-based clone detection approach that transforms code into multiple representations for robust detection. *CloneWorks* (Svajlenko and Roy, 2017): A tool that applies modified Jaccard similarity for efficient and scalable clone detection in large projects.

Additionally, we compare with six deep learning-based code clone detectors, namely *RtvNN* (White et al., 2016): A recurrent neural network model that encodes token sequences into vector representations to detect code clones. *CDLH* (Wei and Li, 2017): A deep hash learning-based model designed for cross-language code clone detection, aligning code semantics across languages. *TBCNN* (Mou et al., 2016): A tree-based convolutional neural network model that extracts structural information from AST to identify clones. *ASTNN* (Zhang et al., 2019b): A clone detection model that splits source code into statement trees and applies recurrent neural networks for similarity analysis. *SCDetector* (Wu et al., 2020b): A semantic code clone detector that leverages graph neural networks to capture both structural and semantic similarities. *FCCA* (Hua et al., 2020): A feature-based approach combining CNNs and LSTMs for detecting clones, particularly in semantically complex scenarios. To better cover recent progress, we further include *CC2Vec* (Dou et al., 2024) as a recent baseline. *CC2Vec* leverages typed-token representations and contrastive learning to capture semantic similarity between code fragments.

Lastly, we evaluate our approach against two advanced Large Language Models (LLMs) as semantic baselines for clone detection. Specifically, we consider *GPT-3.5-turbo* (Ouyang et al., 2022), a cost-effective model optimized for fast inference, and *GPT-4* (Achiam et al., 2023), which offers enhanced reasoning capabilities and improved performance on complex code understanding tasks. In our evaluation, the LLMs are queried in a zero-shot setting to determine whether two given code fragments constitute a clone pair.

For each method, we use the parameter settings recommended in the respective papers to ensure a fair and reliable comparison, with our results closely matching those reported in previous studies.

4.1.4. Metrics

We make use of the following widely used metrics to measure the detection performance of *Gitor*. Precision is defined as $P = TP/(TP + FP)$. Recall is defined as $R = TP/(TP+FN)$. F1 is defined as $F1 = 2 * P * R/(P+R)$. Among them, *true positive* (TP) represents the number of samples correctly classified as clone pairs, *false positive* (FP) represents the number of samples incorrectly classified as clone pairs, and *false negative* (FN) represents the number of samples incorrectly classified as non-clone pairs.

4.2. RQ1: Effectiveness of Keyword-based Detection

As introduced earlier, our system *Gitor* extracts individual information from code samples, including keywords and code metrics. In this section, we focus solely on using keywords to construct the global graph for code clone detection.

To evaluate the effectiveness of *Gitor* in detecting code clones using only keywords, we conducted experiments by varying the cosine similarity threshold and the embedding dimension. The cosine similarity threshold was set to 0.50, 0.60, 0.70, 0.80, and 0.90, while the embedding dimension was set to 16, 32, 64, and 128. The results are illustrated in Table 3, including the recall, precision, and F1 scores of our experiment on the dataset. As for the measurement of precision, unlike machine learning–based detectors that evaluate precision directly on labeled test sets, our graph-based approach inevitably generates many clone pairs beyond the annotated ones. Therefore, following prior works (Wang et al., 2018; Saini et al., 2018), we randomly sample 400 clone pairs from clone reports in each tool and conduct manual analysis to validate them. Each clone pair is checked independently by two experts. If there is a conflict, a final decision will be made after discussion with another expert. The principle rule for judging is based on the overall similarity between the two clone fragments and on whether they perform similar functionality.

As shown in Table 3, the performance of keyword-based detection varies notably across different similarity thresholds and embedding dimensions. First, as the threshold increases, the precision improves while the recall decreases. This is reasonable since a higher similarity threshold ensures that detected clone pairs exhibit stronger similarity, leading to higher precision. However, it also causes some clone pairs with slightly lower similarity to be filtered out, thereby reducing recall. Second, different embedding dimensions also affect detection performance. Lower-dimensional embeddings (*e.g.*, 16 and 32) tend to retain more global structural information, leading to higher recall, but they may fail to capture finer details, resulting in lower precision. In contrast, higher-dimensional embeddings (*e.g.*, 64 and 128) capture more nuanced differences, thereby improving precision but at the cost of recall.

4.3. RQ2: Effectiveness of Code Metrics-based Detection

In addition to keyword-based clone detection, we also evaluated *Gitor*'s effectiveness using code metrics. Specifically, we examine the performance of using all available code metrics and eight code metric groups in clone detection.

For the experiment of all code metrics, we followed the same methodology as in the keyword-based approach, extracting features from code samples and constructing a global graph representation. The cosine similarity threshold

Table 3
Detection performance of *Gitor* with keywords.

Measure	Cosine = 0.50				Cosine = 0.60				Cosine = 0.70	
	16	32	64	128	16	32	64	128	16	32
T-1 Recall	1	1	1	1	1	1	1	1	1	1
T-2 Recall	1	1	1	1	1	1	1	1	1	1
VST-3 Recall	0.999	0.999	0.993	0.992	0.997	0.998	0.972	0.990	0.991	0.974
ST-3 Recall	0.987	0.961	0.914	0.911	0.947	0.886	0.839	0.835	0.893	0.819
MT-3 Recall	0.907	0.841	0.752	0.709	0.819	0.714	0.599	0.553	0.690	0.542
Type-4 Recall	0.375	0.313	0.200	0.157	0.264	0.179	0.108	0.008	0.145	0.081
Precision	0.820	0.843	**0.917**	0.896	0.860	0.892	**0.947**	0.930	0.888	0.934
F1	**0.682**	0.663	0.610	0.573	**0.641**	0.585	0.527	0.496	**0.560**	0.494
Measure	**Cosine = 0.80**				**Cosine = 0.90**				**Cosine = 0.70**	
	16	32	64	128	16	32	64	128	64	128
T-1 Recall	1	1	1	1	1	1	1	1	1	1
T-2 Recall	1	1	1	1	1	1	1	1	1	1
VST-3 Recall	0.966	0.937	0.924	0.912	0.882	0.830	0.827	0.818	0.964	0.964
ST-3 Recall	0.804	0.662	0.625	0.625	0.657	0.510	0.497	0.492	0.745	0.742
MT-3 Recall	0.494	0.329	0.258	0.230	0.276	0.141	0.119	0.114	0.430	0.390
Type-4 Recall	0.062	0.027	0.015	0.012	0.017	0.005	0.004	0.003	0.047	0.036
Precision	0.932	0.974	**0.988**	0.983	0.983	0.991	**0.993**	0.991	**0.971**	0.959
F1	**0.469**	0.399	0.369	0.358	**0.377**	0.320	0.311	0.309	0.445	0.425

Table 4
Detection performance of *Gitor* with All Metrics.

Measure	Cosine = 0.50				Cosine = 0.60				Cosine = 0.70	
	16	32	64	128	16	32	64	128	16	32
T-1 Recall	1	1	1	1	1	1	1	1	1	1
T-2 Recall	1	1	1	1	1	1	1	1	1	1
VST-3 Recall	1	1	1	1	1	1	1	1	1	1
ST-3 Recall	1	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.999	0.998
MT-3 Recall	0.995	0.994	0.992	0.993	0.990	0.990	0.981	0.982	0.970	0.975
Type-4 Recall	0.851	0.892	0.789	0.766	0.749	0.820	0.649	0.617	0.584	0.701
Precision	0.710	0.696	0.743	**0.744**	0.739	0.721	**0.787**	0.780	0.778	0.761
F1	0.797	0.797	**0.801**	0.796	0.789	**0.796**	0.785	0.771	0.759	**0.786**
Measure	**Cosine = 0.80**				**Cosine = 0.90**				**Cosine = 0.70**	
	16	32	64	128	16	32	64	128	64	128
T-1 Recall	1	1	1	1	1	1	1	1	1	1
T-2 Recall	1	1	1	1	1	1	1	1	1	1
VST-3 Recall	1	0.999	1	0.999	0.999	0.999	0.999	0.999	1	1
ST-3 Recall	0.995	0.966	0.990	0.990	0.961	0.975	0.916	0.922	0.998	0.999
MT-3 Recall	0.896	0.916	0.751	0.764	0.628	0.667	0.360	0.357	0.934	0.939
Type-4 Recall	0.355	0.501	0.208	0.193	0.100	0.198	0.045	0.039	0.441	0.416
Precision	0.842	0.818	0.879	**0.883**	0.943	0.891	0.947	**0.959**	**0.835**	0.826
F1	0.690	**0.741**	0.608	0.603	0.533	**0.589**	0.432	0.428	0.728	0.715

was set to 0.50, 0.60, 0.70, 0.80, and 0.90, while the embedding dimension was varied across 16, 32, 64, and 128. Through the results in Table 4, we find that the performance of all metrics is similar to those observed for keywords in Table 3. However, there are some differences between all metrics and keywords. In terms of recall, all metrics consistently achieves higher recall for Type-3 and Type-4 clones compared to keywords. For example, at a cosine threshold of 0.50 and a keyword size of 128, the Type-4 Recall for all metrics is 0.766, significantly surpassing the 0.157 achieved by keywords. This highlights the strength of all metrics in detecting semantic similarities in complex code clones. Nevertheless, this improvement in recall comes at the cost of precision and F1 score. For instance, at a cosine threshold of 0.90 and a keyword size of 128, the precision of all metrics is 0.959, slightly lower than the 0.991 achieved by keywords. Similarly, the F1 scores of all metrics are generally lower across various configurations, particularly at higher thresholds.

To further assess the effectiveness of *code metrics* in clone detection, we systematically categorized 27 distinct code metrics into eight functional groups (*i.e.,* as shown in Table 2). For each metric group, we developed a comprehensive global sample graph to represent the code structure. We then extracted characteristic feature vectors from these graphs and employed cosine similarity calculations between code samples to evaluate the clone detection accuracy.

In our experimental evaluation, we conducted a comprehensive analysis by varying two key parameters: the cosine similarity threshold (*i.e.,* ranging from 0.5 to 0.99, with specific values at 0.5, 0.6, 0.7, 0.8, 0.9, 0.93, 0.96, and 0.99) and the feature vector dimensions (*i.e.,* 16, 32, 64, and 128). The choice of these threshold intervals was deliberate: initially, we analyzed the thresholds in increments

Table 5
Performance of different metric groups with precision ≥ 0.90.

Combination of Metrics	Precision	Cosine	Dimension
Complexity	0.905	0.96	128
Code Struct	0.917	0.93	64
Ex Class References	0.904	0.90	16
Global Call	0.918	0.96	128
Halstead Metrics	0.916	0.99	32
Literal Metrics	0.903	0.93	16
Local Call	**0.933**	**0.99**	**128**
Variables Methods	0.911	0.96	16

Table 6
Performance of different metric groups with precision ≥ 0.95.

Combination of Metrics	Precision	Cosine	Dimension
Complexity	0.964	0.99	32
Code Struct	0.955	0.96	32
Ex Class References	0.954	0.96	64
Global Call	-	-	-
Halstead Metrics	-	-	-
Literal Metrics	0.954	0.99	128
Local Call	-	-	-
Variables Methods	**0.972**	**0.99**	**64**

of 0.1 from 0.5 to 0.9. However, we observed that even at a threshold of 0.9, the precision was still suboptimal. To better capture the performance changes at higher similarity levels, we adopted a finer interval of 0.03 for thresholds above 0.9. The results, including recall, precision, and F1 scores of our experiment on the dataset, as well as the performance analysis charts for each combination of code metrics, are now available on our GitHub repository. Each set of charts shows detection performance based on different cosine similarity thresholds ranging from 0.5 to 0.99, evaluated under embedding dimensions of 16, 32, 64, and 128. Given the extensive experimental results and the complexity of Table 2, we primarily focus on cases where precision exceeds 0.90 and 0.95 to provide a clearer view of high-precision detection scenarios. Tables 5 and 6 present the first occurrence where each metric group achieves precision values exceeding 0.90 and 0.95 respectively, documenting the corresponding similarity thresholds, vector dimensions, and precise precision values.

Upon examining Tables 5 and 6, we observe several notable patterns in clone detection performance across different metric combinations. First, most metric combinations can achieve precision above 0.90 with moderate similarity thresholds (*i.e.,* between 0.90 and 0.93), while achieving precision above 0.95 generally requires higher thresholds (*i.e.,* between 0.96 and 0.99). Second, three groups, namely Global Call, Halstead Metrics, and Local Call, failed to achieve precision levels above 0.95 even at the highest similarity threshold of 0.99, indicating their limited effectiveness as standalone metrics for high-precision clone detection. Third, the vector dimensions required for optimal performance vary significantly among different metric combinations, ranging from 16 to 128 dimensions. Notably, certain groups, such as Literal Metrics, require only 16 dimensions to attain superior results.

4.4. RQ3: Effectiveness of Combining Code Metrics with Keywords

In this part, we pay attention to the effectiveness when *keywords* are combined with *code metrics*. Specifically, we aim to measure the performance of clone detection across various similarity thresholds (*i.e.,* 0.5, 0.6, 0.7, 0.8, and 0.9) and feature vector dimensions (*i.e.,* 16, 32, 64, and 128), using the combination of keywords and code metrics.

Based on the experimental results, as shown in Table 7, several critical trends and insights can be summarized. First, combining *code metrics* with keyword-based approaches provides additional improvements in clone detection effectiveness. The combination of *keywords* and *code metrics* achieves higher precision compared to using *keywords* alone. Moreover, the results highlight the importance of embedding size. Higher vector dimensions (*e.g.*, 64 or 128) often lead to better detection performance, especially for complex clone types. Finally, the combined approach offers a trade-off between different metrics. While precision is consistently enhanced, recall tends to drop, which in turn lowers the F1 score. Nevertheless, code metrics provide complementary structural information that enriches the representation space, even when the overall F1 does not necessarily improve. To provide a clearer comparison of the effectiveness of combining keywords with code metrics for clone detection, we visualize the F1-score and precision score across different cosine similarity thresholds and embedding dimensions. The four subplots in Figure 7 illustrate the performance trends for embedding dimensions of 16, 32, 64, and 128. As shown in the figures, combining keywords with code metrics significantly improves precision by capturing complementary features. However, this improvement introduces a trade-off with Recall. While Recall is omitted to maintain visual clarity, our analysis reveals its expected decline at higher thresholds (e.g., dropping to 0.15 at threshold 0.90 for $d = 64$). Nevertheless, the F1-score bars effectively capture this balance, demonstrating that *Gitor* maintains competitive performance at moderate thresholds.

In short, integrating *code metrics* into keyword-based clone detection improves precision and provides complementary information, while also allowing a better balance between precision and recall.

Since the neural variants of *Gitor* rely on learned embeddings, we examine the impact of embedding dimensionality across the three previously evaluated representation settings. For brevity, *Gitor-2L* serves as the representative classifier, with network depth comparisons deferred to RQ4. Table 8 shows that the performance of *Gitor-2L* is sensitive to dimensionality. Specifically, a 16-dimensional embedding is suboptimal, whereas larger embedding dimensions generally yield stronger results. Furthermore, the joint representation of keywords and code metrics consistently achieves the highest performance across all dimensions, underscoring the advantage of this combination. Notably, optimal performance

Table 7
Detection performance of *Gitor* with Both.

Measure	Cosine = 0.50				Cosine = 0.60				Cosine = 0.70	
	16	32	64	128	16	32	64	128	16	32
T-1 Recall	1	1	1	1	1	1	1	1	1	1
T-2 Recall	1	1	1	1	1	1	1	1	1	1
VST-3 Recall	1	0.998	0.998	0.998	1	0.998	0.997	0.997	0.996	0.961
ST-3 Recall	0.984	0.959	0.974	0.944	0.970	0.923	0.911	0.901	0.895	0.863
MT-3 Recall	0.919	0.832	0.804	0.703	0.829	0.727	0.650	0.558	0.680	0.568
Type-4 Recall	0.448	0.282	0.198	0.175	0.311	0.165	0.099	0.083	0.184	0.077
Precision	0.807	0.839	0.886	**0.902**	0.849	0.889	0.930	**0.937**	0.892	0.938
F1	**0.718**	0.646	0.613	0.584	**0.662**	0.580	0.533	0.503	**0.581**	0.500

Measure	Cosine = 0.80				Cosine = 0.90				Cosine = 0.70	
	16	32	64	128	16	32	64	128	64	128
T-1 Recall	1	1	1	1	1	1	1	1	1	1
T-2 Recall	1	1	1	1	1	1	1	1	1	1
VST-3 Recall	0.932	0.951	0.941	0.934	0.899	0.878	0.850	0.833	0.978	0.976
ST-3 Recall	0.824	0.730	0.687	0.659	0.701	0.560	0.491	0.474	0.816	0.788
MT-3 Recall	0.508	0.339	0.257	0.205	0.269	0.150	0.097	0.084	0.448	0.377
Type-4 Recall	0.082	0.025	0.012	0.009	0.019	0.005	0.002	0.001	0.040	0.033
Precision	0.943	0.976	0.986	**0.989**	0.983	0.992	**0.996**	**0.996**	0.962	**0.969**
F1	**0.488**	0.404	0.370	0.352	**0.379**	0.326	0.303	0.298	0.447	0.442

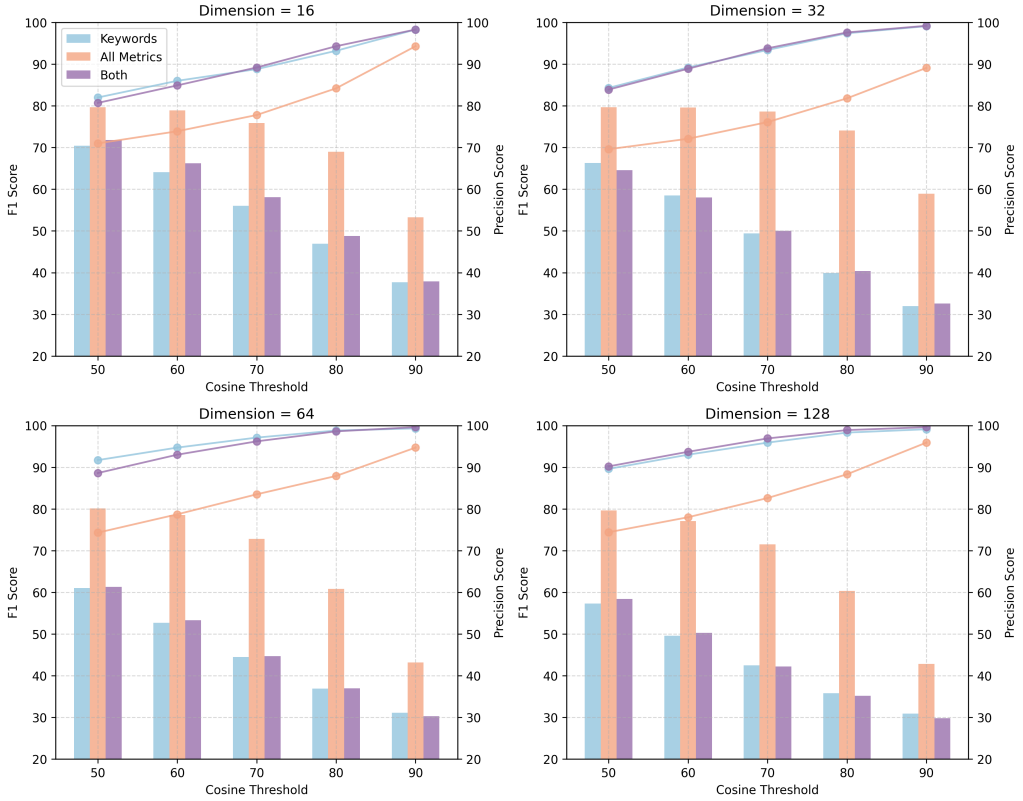


Figure 7: Clone detection performance across embedding dimensions (*i.e.,* 16, 32, 64, 128) and cosine thresholds (*i.e.,* 0.50–0.90). Bars show F1 and lines indicate precision, comparing *keywords*, *metrics*, and their combination.

is achieved at 64 dimensions using the joint representation, while increasing to 128 yields no consistent improvements across the settings.

4.5. RQ4: Comparative with Other Detectors

Our proposed *Gitor* model supports two distinct methods for code clone detection. The first method is based on computing the cosine similarity between the vector representations of code samples, enabling straightforward and efficient clone detection. The second method utilizes neural networks as classifiers, which process pairs of code vectors to determine whether they represent clone pairs. For the neural network-based method, we introduce three variants: *Gitor-1L*, *Gitor-2L*, and *Gitor-3L*. Each variant employs an increasing number of fully connected layers with ReLU

Table 8
Overall F1 of *Gitor-2L* under different embedding dimensions for the three representation settings.

Representation	Embedding Dimension			
	16	32	64	128
Keywords	0.948	0.964	0.968	0.973
All Metrics	0.952	0.967	0.978	0.972
Both	0.967	0.979	0.986	0.983

Table 9
Results of conventional methods and *Gitor* on the benchmark dataset.

Methods	Recall						Precision
	T1	T2	VST3	ST3	MT3	T4	
SourcererCC	1	0.97	0.93	0.6	0.05	0	0.98
CCFinder	1	0.93	0.62	0.15	0.01	0	0.72
NiCad	1	0.99	0.98	**0.93**	0.01	0	**0.99**
Deckard	0.6	0.58	0.62	0.31	0.12	0.01	0.35
CCAligner	1	0.99	0.97	0.7	0.1	0	0.8
Oreo	1	0.99	1	0.89	0.3	0.01	0.89
LVMapper	0.99	0.99	0.98	0.81	0.19	0	0.58
NIL	0.99	0.96	0.93	0.67	0.1	0	0.94
Siamese	1	0.96	0.85	0.59	0.14	0	0.98
CloneWorks	1	0.98	0.88	0.64	0.15	0	0.96
Gitor	1	**1**	**0.98**	0.82	**0.45**	**0.04**	0.96

activations to improve detection performance and capture complex relationships in the code vectors. Because these two detection settings follow different evaluation protocols, the reported metrics in this section are presented at different levels of granularity. Accordingly, Table 9 reports per-type recall together with overall precision for the setting evaluated via cosine similarity, whereas Table 11 reports overall recall, precision, and F1 for the neural network classifiers.

Conventional Clone Detectors. In order to comprehensively evaluate *Gitor*'s performance, we compare the performance of *Gitor*'s clone detection against the latest versions of several publicly available clone detection tools, such as *SourcererCC* (Sajnani et al., 2016), *CCFinder* (Kamiya et al., 2002), *NiCad* (Roy and Cordy, 2008), *Deckard* (Jiang et al., 2007), *CCAligner* (Wang et al., 2018), *Oreo* (Saini et al., 2018), *LVMapper* (Wu et al., 2020a), *NIL* (Nakagawa et al., 2021), *Siamese* (Ragkhitwetsagul and Krinke, 2019), and *CloneWorks* (Svajlenko and Roy, 2017). Since many widely used clone detection tools (e.g., *SourcererCC* and *NiCad*) use 0.7 as the threshold for clone identification, we also adopt 0.7 in RQ4 to maintain a fair comparison setting. Under this common threshold, our goal is not to choose the configuration with the single best value on one metric, but to evaluate the final configuration of *Gitor* used for comparison. Although the all-metrics setting achieves higher F1 scores in some cases, it yields noticeably lower precision than the combined setting. By contrast, in some cases, the keywords-only setting attains slightly higher precision, but it does not exploit the complementary structural information provided by code metrics. Therefore, for RQ4 we use the combined setting of keywords and code metrics. Within this combined setting, we use embedding dimension 64 as a representative configuration that delivers strong overall performance without introducing a larger embedding size. As Table 9 shows, *Gitor* outperforms every other tool on most of the clone categories, except for ST3. Although *NiCad* performs the best on ST3, *Gitor*'s performance on ST3 clone is still quite comparable to the state-of-the-art since there is only an 11 percent difference. The recall results are promising since they suggest that, in addition to recognizing easier-to-find clones like T1, T2, and VST3, *Gitor* also detects clones that other tools miss. In comparison to other methods, where *Oreo*'s highest recall is 0.3, 0.45 recall in the MT3 category is a significant improvement. Table 9 also shows the precision results of all tools. The precision of *Gitor* is 0.96, with only *SourcererCC*, *Siamese*, and *NiCad* outperforming it by 3 percent.

Table 10
Sampled per-type precision, recall, and F1 of *Gitor* under the final comparison configuration on the benchmark dataset.

Clone Type	Sample Size	Precision	Recall	F1
T1	50	0.989	1	0.994
T2	50	0.977	1	0.988
VST3	50	0.930	0.976	0.952
ST3	100	0.891	0.822	0.855
MT3	100	0.739	0.450	0.559
T4	150	0.588	0.044	0.082

Note: Sample sizes were set by stratified sampling, with larger samples for ST3, MT3, and T4 to better characterize harder clone types.

To better understand *Gitor*'s strengths and weaknesses across different clone types, the comparative evaluation is further supplemented with a sampled per-type analysis based on the configuration in Table 9. In our setting, clone-type recall can be obtained directly from the benchmark labels, whereas precision is less straightforward to compute. Unlike supervised learning approaches that evaluate precision directly on labeled test pairs, *Gitor* reports many additional candidate clone pairs beyond the benchmark. Consequently, many detected pairs lack ground-truth labels, and their correctness cannot be computed directly from the benchmark alone. Thus, following the RQ1 methodology, we conduct a sampled per-type precision analysis and report the corresponding F1 alongside recall. Table 10 shows that *Gitor* remains highly effective on syntactically similar clone types, while performance degrades on more challenging semantic clones.

Deep Learning-Based Detectors. To extensively evaluate the performance of *Gitor*'s neural network-based variants, we compare the performance of clone detection against the state-of-the-art deep learning-based code clone detectors, such as *RtvNN* (White et al., 2016), *CDLH* (Wei and Li, 2017), *TBCNN* (Mou et al., 2016), *ASTNN* (Zhang et al., 2019b), *SCDetector* (Wu et al., 2020b), and *FCCA* (Hua et al., 2020). We further include the recent baseline *CC2Vec-3L* (Dou et al., 2024), as it is the classifier-based variant of *CC2Vec* and thus provides a more direct comparison with *Gitor*'s neural models under the same experimental protocol. Table 11 presents the Recall, Precision, and F1 scores of our proposed method alongside these comparative deep learning-based code clone detectors on the dataset.

Gitor-1L, *Gitor-2L*, and *Gitor-3L* differ in network depth, as described in Section 3.4.2. For *Gitor*'s neural network-based variants, we observe clear benefits in detecting more challenging clones. Compared with cosine similarity alone, the neural network–based models achieve higher overall recall, indicating a stronger capability to detect clones beyond linear similarity. Notably, further increasing the number of fully connected layers to three in *Gitor-3L* results in similar F1 scores, indicating limited benefits from additional layer complexity.

Among the evaluated code clone detection methods, *RtvNN*, *CDLH*, *TBCNN*, and *ASTNN* are all based on AST structures. Specifically, *RtvNN* uses an RNN model to encode both source code tokens and ASTs for clone detection, although it is sensitive to minor changes in tree structure, leading to the F1 score of only 1%. The other AST-based detectors, however, achieve stronger results. For instance, *ASTNN* achieves the F1 score of 93%. Additionally, *SCDetector* and *FCCA* represent graph-based approaches to semantic clone detection, achieving strong recall rates of 94% and 92%, respectively, which further demonstrates the effectiveness of graph-based modeling for this task. *CC2Vec-3L*, as a recent contrastive learning-based baseline built upon typed-token representations, also achieves strong performance with a recall of 97%, a precision of 94%, and an F1 score of 95%. This result indicates that recent token-level representation learning methods can provide highly competitive clone detection performance. Nevertheless, *Gitor-2L* and *Gitor-3L* still achieve higher overall F1 scores, suggesting that modeling global relationships through keywords and code metrics provides complementary benefits beyond token-level representation learning alone. Compared with these deep learning-based detectors, *Gitor-2L* and *Gitor-3L* achieve highly competitive performance while remaining conceptually simpler and more scalable. Although *Gitor-2L* and *Gitor-3L* achieve an overall F1 of 0.99 on BigCloneBench, this result should be understood in the context of the current evaluation setting and benchmark characteristics. Specifically, these classifier results are obtained on a function-disjoint train/test split with balanced clone and non-clone pairs, which helps reduce the risk of data leakage and provides a basis for comparison across classifier-based methods. At the same time, such a very high overall score may also be influenced by the composition of the benchmark dataset and should not be over-generalized beyond this setting. Therefore, we interpret the reported 0.99 F1 as evidence that the learned representations and classifier are highly effective under this benchmark setting, rather than as a claim that code clone detection is solved in general.

LLM–Based Detectors. Recent Large Language Models such as *GPT-3.5-turbo* (Ouyang et al., 2022) and *GPT-4* (Achiam et al., 2023) have shown remarkable capabilities in code understanding, and we include them as baselines for comparison. We leveraged their APIs to perform pairwise clone detection using a unified prompt and a controlled inference configuration. Specifically, for each code pair, we provide the following prompt to the LLMs: *"Please examine the following two code snippets and determine whether they should be considered code clones (including Type-1 to Type-4 clones). Respond with 'yes' if they are clones, and 'no' if they are not. Output only 'yes' or 'no' without additional explanation."* To ensure reproducibility, all LLM queries were executed with the temperature set to zero, and nucleus sampling was disabled by setting top-p to 1. This zero-shot setting is intended to provide a simple and reproducible reference rather than a full exploration of the capabilities of modern LLMs. More advanced prompting or task-specific adaptation may further improve LLM-based clone detection and are left for future work.

Table 11
Results of *Gitor* variants and comparative systems on the experimental dataset.

Methods	Recall	Precision	F1
RtvNN	0.01	0.95	0.01
CDLH	0.74	0.92	0.82
TBCNN	0.81	0.90	0.85
ASTNN	0.94	0.92	0.93
SCDetector	0.92	0.97	0.94
FCCA	0.92	0.98	0.95
CC2Vec-3L	0.97	0.94	0.95
GPT-3.5	0.65	0.95	0.77
GPT-4	0.81	0.96	0.88
Gitor-1L	0.89	0.89	0.89
Gitor-2L	0.98	**0.99**	**0.99**
Gitor-3L	**0.99**	0.99	0.99

As shown in Table 11, Large Language Models show promising potential in the task of code clone detection, particularly *GPT-4* which recorded a recall of 0.81, precision of 0.96, and an F1 score of 0.88. Although they are not yet the optimal solution, considering that these models are general-purpose and not specifically designed for clone detection, their performance under the unified zero-shot prompting setting is already encouraging. At the same time, these results should be interpreted as reference results under this simple setting, rather than as the upper bound of LLM performance on this task.

The experimental results demonstrate that *Gitor* serves as a reliable and accurate code clone detector capable of effectively identifying various types of clones. Through both cosine similarity calculation and neural network-based classification methods, our model validates effectiveness as a comprehensive code clone detection technique.

4.6. RQ5: Scalability

In this section, we focus on the runtime performance of *Gitor*. As mentioned before, scalability is an important requirement for clone detection methods, and *Gitor* is designed as a scalable clone detection system. Our final experiment assesses this capability by analyzing one million randomly selected code pairs from the dataset and measuring the runtime overhead of *Gitor* in comparison to other detection systems. For completeness, we report the runtime of both *Gitor* with cosine similarity and its neural network-based

Table 12
Time performance of code clone detectors on analyzing one million code pairs.

Tools	Conventional Methods									Deep Learning-based Methods							Ours			
	Sour.	CCF.	NiCad	Deck.	CCAl.	Oreo	LVM.	NIL	Tamer	RtvNN	CDLH	TBCNN	ASTNN	SCDet.	FCCA	CC2-3L	Gitor	1L	2L	3L
Prediction	16s	23s	14s	72s	10s	15s	11s	9s	10s	35s	90s	86s	2,894s	139s	91s	83s	11s	113s	117s	115s
Training	0s	0s	0s	0s	0s	0s	0s	0s	0s	5,206s	45,317s	41,168s	16,096s	2,937s	56,769s	4,871s	0s	2,125s	2,132s	2,143s

Abbreviations: Sour. = SourcererCC; CCF. = CCFinder; NiC. = NiCad; Deck. = Deckard; CCAl. = CCAligner; LVM. = LVMapper; SCDet. = SCDetector; CC2-3L = *CC2Vec-3L*; Gitor = *Gitor* with cosine similarity; 1L–3L = variants of *Gitor* with 1–3 layers.

variants. However, *GPT-3.5* and *GPT-4* were excluded from this scalability evaluation. The primary reason for this exclusion is the cost and computational constraints associated with using GPT models. Given that processing one million code pairs would require an enormous number of API requests, the associated financial and computational overhead would be prohibitive.

To ensure a fair comparison and reliable runtime measurements, all experiments are executed on the identical hardware environment described in Section 4.1.2. Each tool is executed three times, and we report the average runtime as the final result in Table 12. Deep learning-based methods, including *RtvNN*, *CDLH*, *TBCNN*, *ASTNN*, *SCDetector*, *FCCA*, *CC2Vec-3L*, *Gitor-1L*, *Gitor-2L*, and *Gitor-3L*, require model training before detecting code clones. Specifically, their runtime overheads are divided into training time and prediction time. Notably, *Gitor-1L*, *Gitor-2L*, and *Gitor-3L* are distinct, as their input is the vector representation of code pairs. Consequently, their training time must also account for the time needed by *Gitor* to construct the global graph and generate vectors using node embedding. In contrast, conventional methods, such as *SourcererCC*, *CCFinder*, *NiCad*, *Deckard*, *CCAligner*, *Oreo*, *LVMapper*, *NIL*, and *Tamer*, typically do not require a training phase, as they rely on token, syntax, or simple structural analysis techniques for clone detection. Specifically, their runtime overhead consists only of prediction time, which is the time taken to analyze and compare code pairs based on predefined patterns, metrics, or token representations. Similarly, *Gitor* with cosine similarity does not involve supervised model training. Therefore, its training time is reported as 0s in Table 12, while its prediction time includes the overall runtime of graph construction, node embedding, and similarity-based detection. In terms of prediction time, the token-based method *NIL* achieves the lowest runtime by using a simple N-gram representation of token sequences for clone detection. In comparison, *Gitor* and its neural network-based variants, *Gitor-1L*, *Gitor-2L*, and *Gitor-3L*, require slightly more runtime to process the same code pairs. This additional time is reasonable, as *Gitor* performs multiple vector-based similarity calculations to determine clone pairs. This process enables *Gitor* to perform detailed analysis of vector information containing various types of code metrics and keywords, though it does introduce some latency. Overall, *Gitor* performs graph construction and embedding once for the dataset, and per-pair detection is reduced to vector comparison that is linear in the embedding dimension, thereby avoiding expensive tree- or graph-matching procedures.

The *Gitor* series models, which include *Gitor-1L*, *Gitor-2L*, and *Gitor-3L*, build on *Gitor* by adding one, two, and three fully connected layers, respectively, alongside ReLU activation functions for enhanced non-linear feature learning. As shown in Table 12, *Gitor* with cosine similarity achieves efficient runtime, requiring only 11 seconds to analyze one million code pairs without supervised training. This makes it faster than most conventional detectors in prediction time. The *Gitor* series remains highly competitive in scalability compared to alternative deep learning approaches. *Gitor-1L* completes training in 2,125 seconds with a detection time of 113 seconds, while *Gitor-2L* requires 2,132 seconds for training and 117 seconds for detection. *Gitor-3L* exhibits marginally longer training duration at 2,143 seconds but maintains competitive detection speed at 115 seconds. Compared with the recent baseline *CC2Vec-3L*, the *Gitor* series exhibits a different trade-off between training and prediction time: *CC2Vec-3L* achieves faster prediction, whereas *Gitor*'s neural variants require substantially less training time. More broadly, among the deep learning-based methods considered in Table 12, the *Gitor* series also maintains the lowest training overhead, even lower than *SCDetector*, which requires 2,937 seconds. For *ASTNN*, predicting one million code pairs takes 2,894 seconds, approximately 24 times slower than *Gitor-2L*, which has the longest prediction time within the *Gitor* series. Although the *Gitor* series takes more time for prediction compared to *CDLH*, *TBCNN*, and *FCCA*, they show significant advantages in training efficiency. *Gitor-2L*, the model with the longest training time in the series, is about 21 times faster than *CDLH*, 19 times faster than *TBCNN*, and 26 times faster than *FCCA*. Furthermore, although *RtvNN* achieves the shortest detection time among deep learning-based methods, its training phase is considerably lengthy—not only requiring more time than the *Gitor* series but also exceeding that of SCDetector.

In summary, *Gitor* with cosine similarity achieves prediction efficiency comparable to the fastest conventional clone detectors, requiring only 11 seconds to analyze one million code pairs. It is faster than most conventional methods and all compared deep learning-based baselines in pure prediction time. Meanwhile, although the neural variants introduce additional prediction overhead due to their fully connected layers, they maintain favorable scalability compared with most deep learning-based methods and require substantially less training time. The development of such efficient detection systems is crucial, as they minimize resource usage and enable the analysis of code clones at scale.

5. DISCUSSION

5.1. Threats to Validity

Internal validity: Our approach employs a node embedding technique to represent the nodes of the global graph as vectors. However, the characteristics of the chosen embedding method may limit the system's ability to capture deeper semantic information. To mitigate this threat, we fix the node embedding method to ProNE across all experiments. This design keeps the evaluation focused on the proposed global graph construction, rather than on interactions between graph construction and different embedding algorithms. Nevertheless, the choice of node embedding method may still influence the learned representations. Evaluating *Gitor* with more recent graph embedding methods, including force-directed and neural aggregation methods, remains an important direction for future work. Furthermore, the results of runtime and performance may be subject to variations due to changes in machine states, such as CPU and GPU utilization. To reduce this threat, we repeated each experiment three times and reported the average results. **External validity:** The distribution of clone types in the benchmark dataset can introduce bias into overall performance metrics. To address this threat, we not only report overall precision and F1 but also provide recall results for each clone type (*i.e.,* from Type-1 to Type-4). Accordingly, the very high overall scores should be interpreted with respect to this benchmark setting. In addition, our main evaluation is conducted on BigCloneBench, which focuses on Java and function-level clone detection. Although BigCloneBench enables comparison with prior studies, relying on a single primary benchmark may limit the generalizability of the findings. To partially address this issue, we further conduct an additional validation on GoogleCodeJam in Section 5.3. The results provide complementary evidence that *Gitor* can be applied beyond BigCloneBench. **Construct validity:** The selection of code metrics in this study may affect the validity of the results. Specifically, the inclusion of highly correlated metrics describing similar information could lead to redundancy, impacting the system's performance. We mitigate the threat by applying a correlation threshold to filter out redundant metrics. Another potential threat is that the edge weights from keywords and metrics may be on different scales. We preserved their original values to retain semantics and observed no negative effects in our experiments, while more sophisticated normalization or weighting schemes could be explored in future work. **Conclusion validity:** One potential threat is that the results may be influenced by random variations in training the neural network, which could lead to unstable conclusions. Following standard practices in empirical software engineering, we conducted the experiments under the same hardware environment and report average results over multiple runs.

5.2. Why Gitor outperforms the other approaches

First, existing clone detection tools (*e.g.*, *CCFinder* (Kamiya et al., 2002) and *SourcererCC* (Sajnani et al., 2016)) focus on analyzing code samples individually without considering the underlying connection among code samples. However, *Gitor* considers the connection among different code samples by extracting the *individual information* of a code sample as its representation. Second, *Gitor* integrates both keywords and code metrics into its global graph construction. By combining these two data types, *Gitor* constructs richer and more informative graph representations of code, enabling it to capture both syntactic and semantic characteristics and significantly improve its ability to identify clones across various levels of similarity. Finally, we introduce neural network classifiers to evaluate pairs of code vectors, constructing a more advanced and comprehensive code clone detector.

Table 13
Comparison results on the GoogleCodeJam benchmark.

Method	Recall	Precision	F1
RtvNN	0.90	0.20	0.33
CDLH	0.70	0.46	0.55
TBCNN	0.89	0.91	0.90
ASTNN	0.87	**0.95**	0.91
SCDetector	0.87	0.81	0.82
FCCA	0.90	**0.95**	0.92
CC2Vec-3L	0.95	0.93	0.94
Gitor	0.75	0.64	0.69
Gitor-2L	**0.96**	0.94	**0.95**

5.3. Additional Validation on GoogleCodeJam

To further examine whether the effectiveness of *Gitor* is limited to BigCloneBench, we conduct an additional validation on GoogleCodeJam. Different from BigCloneBench, GoogleCodeJam is derived from programming contest submissions, where programs solving the same problem are usually treated as clone pairs and programs from different problems are treated as non-clone pairs. Therefore, this benchmark provides a complementary setting for assessing the applicability of *Gitor* beyond the primary benchmark.

Table 13 reports the additional validation results on GoogleCodeJam. *Gitor* with cosine similarity achieves an F1 score of 0.69, indicating that the unsupervised similarity-based variant can still capture part of the clone relationships under a different benchmark setting. Compared with this variant, *Gitor-2L* substantially improves all three metrics and achieves a recall of 0.96, a precision of 0.94, and an F1 score of 0.95. Among the compared methods, *Gitor-2L* obtains the highest recall and F1 score, while its precision remains close to the best precision achieved by *ASTNN* and *FCCA*. These results provide complementary evidence that the proposed global sample graph can support clone detection beyond BigCloneBench.

5.4. Lessons

Our experiments show that code structure performs the best among the selected eight code metrics groups. We also observe that in the second detection method of *Gitor*, due to the use of neural networks as classifier, the detection

performance is better than that of the first method by simply calculating cosine similarity to detect clones. For *Gitor*'s neural network-based variants, our experiments showed that increasing the number of fully connected layers to three (*i.e., Gitor-3L*) did not lead to a noticeable improvement in performance. With two fully connected layers (*i.e., Gitor-2L*), the F1 score achieves performance comparable to other widely used deep learning methods. Therefore, for effective clone detection using code metrics, we recommend using the code structure metrics group. For practitioners seeking a balance between detection performance and computational efficiency, the *Gitor-2L* method with two fully connected layers is suggested, as it achieves comparable performance to other advanced deep learning approaches.

Finally, with respect to the two motivating challenges introduced at the beginning of this work, our study confirms their resolution. The first challenge was how to build a global graph that preserves code details. We addressed this by integrating keywords and code metrics into Gitor's design, and our experiments show that this approach effectively retains both syntactic and semantic information. The second challenge was how to leverage the global graph for efficient and accurate clone detection. We addressed this by providing two methods: cosine similarity for scalability and a neural-network classifier for complex semantic clones. Our experiments shows that both methods achieve strong performance.

6. RELATED WORK

The large number of clone detection methods currently proposed can be divided into five categories: text-based methods, token-based methods, tree-based methods, graph-based methods, and metrics-based methods.

The similarity between two code snippets is measured in the form of text or strings for the text-based methods (Johnson, 1994; Ducasse et al., 1999; Roy and Cordy, 2008). (Johnson, 1994) proposes a fingerprinting technique for detecting code clones. (Ducasse et al., 1999) develops a language-independent method for detecting similar codes using only line-based string matching. These two techniques, however, do not support Type-3 clone detection. To detect more types of clones, *Nicad* (Roy and Cordy, 2008) introduces a two-stage approach that consists of i) identifying and normalizing potential clones using flexible pretty-printing and ii) computing similarity by simply text-line comparison using the longest common subsequence algorithm. Although *Nicad* can detect a number of Type-3 clones, it cannot detect Type-4 clones because it ignores the program semantics of given code samples.

For the token-based techniques (Kamiya et al., 2002; Wang et al., 2023; Sajnani et al., 2016; Wang et al., 2018; Ragkhitwetsagul and Krinke, 2019), tokens are first collected from program code by lexical analysis. *CCFinder* (Kamiya et al., 2002) extracts a token sequence from the input code and converts it into a regular form for finding Type-1 and Type-2 clones using numerous rule-based transformations, and *SourcererCC* (Sajnani et al., 2016) has been developed to support Type-3 clone detection, which is designed to capture the tokens' overlap similarity among multiple approaches for detecting Type-3 clones that are close to being detected. *SourcererCC* (Sajnani et al., 2016) is the most scalable code clone detector, capable of detecting 250 million lines of code. However, token-based detection methods, like text-based approaches, are unable to handle Type-4 clones.

To detect code clones, the tree-based tools (Jiang et al., 2007; Wei and Li, 2017; Zhang et al., 2019b) employ AST as the code representation. *Deckard* (Jiang et al., 2007)'s core idea is to compute characteristic vectors within ASTs and use *Locality Sensitive Hashing* (LSH) to cluster comparable vectors for clone detection. *CDLH* (Wei and Li, 2017) first converts ASTs to binary trees, then uses Tree-LSTM (Tai et al., 2015) to encode these trees into vector representations. Finally, these vectors are utilized to compare distinct codes' similarity. *ASTNN* (Zhang et al., 2019b) separates each huge AST into a sequence of little statement trees, unlike *CDLH* (Wei and Li, 2017). To find semantic code clones, after encoding these statement trees into vectors, a bidirectional RNN model is utilized to construct the final vector representation of a code fragment. These tree-based methods can detect semantic clones, but their scalability is limited due to their long execution times.

In summary, although text-based, token-based, and tree-based detectors use different representations, they generally make clone decisions based on local similarity between independently represented code fragments. In contrast, *Gitor* constructs a global sample graph to model corpus-level relationships among code samples through shared keywords and code metrics.

For graph-based clone detection techniques, program semantics are distilled into graph representations such as program dependency graphs (PDGs) and control-flow graphs (CFGs) (Krinke, 2001; Komondoor and Horwitz, 2001; Wang et al., 2017; Zhao and Huang, 2018; Chen et al., 2014; Wan et al., 2026). Early approaches (Komondoor and Horwitz, 2001; Krinke, 2001) detect clones through subgraph matching. To mitigate the high cost of graph matching, *CCGraph* (Zou et al., 2020) first simplifies PDGs, then applies coarse filtering using numerical features, followed by refinement based on string similarity. Wu et al. (Wu et al., 2020b) abstract CFG semantics into semantic tokens (e.g., via centrality analysis) to enable efficient similarity analysis without explicit subgraph matching. In parallel, learning-based methods treat clone detection as a representation learning or classification problem over graphs. For instance, Hu et al. (Hu et al., 2022) adopt graph neural networks over enriched PDG variants (e.g., FC-PDG with relational GCNs) to capture code context for clone detection. More recently, Liu et al. (Liu et al., 2023) learned representations over code property graphs for source-level functional similarity detection, demonstrating the promise of graph-based learning in capturing semantic relationships among code fragments. Beyond simple similarity matching, to ensure

the robustness and generalizability of structural representations, advanced graph models are increasingly focusing on dynamic topology recovery and structural interaction (Jia et al., 2026), highlighting the broader scientific depth of graph-based pattern recognition. However, while graph-based methods are effective in capturing program semantics, they often face trade-offs between semantic expressiveness and scalability.

Metrics-based clone detection techniques leverage quantitative attributes of source code to measure similarity between code fragments, where metrics can be derived directly from source code or extracted from intermediate representations such as ASTs or program graphs (Balazinska et al., 1999; Patenaude et al., 1999b; Sheneamer and Kalita, 2016; Saini et al., 2021). Early studies employed metrics extracted from ASTs to characterize code fragments and identify clones (Balazinska et al., 1999; Patenaude et al., 1999b). More recent work has extended metric-based clone detection by incorporating richer semantic features and learning-based techniques. For example, Sheneamer et al. (Sheneamer and Kalita, 2016) proposed an efficient metric-driven approach that derives features from ASTs and program dependency graphs, improving scalability while retaining semantic sensitivity. *Oreo* (Saini et al., 2021) further combines software metrics with information retrieval and machine learning models to detect clones, achieving competitive precision and recall on large benchmark datasets. Many commonly used counting metrics exhibit strong correlations with code size, which may reduce robustness when distinguishing semantic clones with substantial structural variation.

Large Language Models (LLMs) have demonstrated remarkable success across various natural language processing tasks (Touvron et al., 2023; Liu et al., 2024; Li et al., 2023; Zheng et al., 2023). Recently, their capabilities have been increasingly applied to software engineering, particularly in code generation and code quality improvement (Jiang et al., 2026b; Liu et al., 2026; Zhang et al., 2026), as well as in refactoring (Ampatzoglou et al., 2026). In the domain of code clone detection, researchers have explored the use of LLMs to identify different clone types in code. For instance, researchers (Moumoula et al., 2024) investigated the use of LLMs and embedding models for cross-lingual code clone detection, highlighting the limitations of LLMs on complex examples and the superior performance of embedding-based classifiers on datasets like XLCoST and CodeNet. Inoue et al. (Inoue and Higo, 2024) enhanced LLM-based code clone detection by fine-tuning models on the FEMPDataset, demonstrating improved accuracy, particularly for detecting clones with low syntactic similarity. Recent work (Jiang et al., 2026a) also suggests that LLMs are promising for clone detection, especially for more complex clones, while still facing challenges such as prompt sensitivity and limited language coverage.

7. CONCLUSION

Building upon the original *Gitor* framework, we make improvements for code clone detection by introducing code metrics and employing a neural network-based detection method. Given a source code repository, we first construct two global sample graphs: one based on keywords and the other on code metrics. These two graphs are then merged into a larger, unified global sample graph. Finally, we apply graph embedding techniques to extract vector representations for all code samples in the repository. These vectors can either be directly used to compute similarity scores between code samples or fed into a neural network to further identify complex code clones. We evaluate *Gitor* on a widely used dataset and compare it with 17 representative clone detectors spanning widely used detectors, deep learning-based baselines, and recently proposed methods, and also include LLM-based approaches. Experimental results show that *Gitor* achieves an F1 improvement of 4%-17% compared with state-of-the-art deep learning detectors. In terms of scalability, *Gitor* achieves prediction efficiency comparable to the fastest conventional detectors and surpasses all compared deep learning-based baselines in prediction time. Moreover, grouped experiments confirm that most code metric combinations yield precision above 0.90, demonstrating that code metrics alone are effective for clone detection.

DATA AVAILABILITY

The source code and dataset supporting this study are available at `https://github.com/tongyangxu12/Gitor_plus`.

ACKNOWLEDGEMENTS

This work was supported in part by the National Natural Science Foundation of China under Grant 62402106; the Natural Science Foundation of Jiangsu Province of China under Grant BK20241272; the "Zhishan" Scholars Programs of Southeast University under Grant 2242026RCB0031; the Fundamental Research Funds for the Central Universities under Grant 2242025K30025; and the Start-Up Research Fund of Southeast University under Grant RF1028623129.

References

Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F.L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al., 2023. Gpt-4 technical report. arXiv preprint arXiv:2303.08774 .

Alomari, H.W., Stephan, M., 2022. Clone detection through srcclone: A program slicing based approach. Journal of Systems and Software 184, 111115.

Ampatzoglou, A., Arvanitou, E.M., Almpanopoulos, S., Mittas, N., Chatzigeorgiou, A., 2026. Ai-assisted code refactoring: Where can it be helpful and where do humans outperform it? Journal of Systems and Software , 112862.

Baker, B.S., 1995. On finding duplication and near-duplication in large software systems, in: Proceedings of the Second Working Conference on Reverse Engineering, IEEE Computer Society, USA. p. 86.

Balazinska, M., Merlo, E., Dagenais, M., Lague, B., Kontogiannis, K., 1999. Measuring clone based reengineering opportunities, in: Proceedings of the 6th International Software Metrics Symposium (ISMS'99).

Bellon, S., Koschke, R., Antoniol, G., Krinke, J., Merlo, E., 2007. Comparison and evaluation of clone detection tools. IEEE Transactions on Software Engineering .

BigCloneBench, 2020. BigCloneBench. https://github.com/clonebench/BigCloneBench.

Chen, K., Liu, P., Zhang, Y., 2014. Achieving accuracy and scalability simultaneously in detecting application clones on android markets, in: Proceedings of the 36th International Conference on Software Engineering (ICSE'14).

Dai, Q., Shen, X., Zhang, L., Li, Q., Wang, D., 2019. Adversarial training methods for network embedding, in: The World Wide Web Conference, pp. 329–339.

Diederik, P.K., 2014. Adam: A method for stochastic optimization. (No Title) .

Dou, S., Wu, Y., Jia, H., Zhou, Y., Liu, Y., Liu, Y., 2024. Cc2vec: Combining typed tokens with contrastive learning for effective code clone detection. Proceedings of the ACM on Software Engineering 1, 1564–1584.

Ducasse, S., Rieger, M., Demeyer, S., 1999. A language independent approach for detecting duplicated code, in: Proceedings of the 1999 International Conference on Software Maintenance (ICSM'99).

German, D.M., Di Penta, M., Gueheneuc, Y.G., Antoniol, G., 2009. Code siblings: technical and legal implications of copying code between applications, in: Proceedings of the 6th International Working Conference on Mining Software Repositories (MSR'09).

Giamphy, E., Guillaume, J.L., Doucet, A., Sanchis, K., 2023. A survey on bipartite graphs embedding. Social Network Analysis and Mining 13, 54.

Grover, A., Leskovec, J., 2016. node2vec: Scalable feature learning for networks, in: Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining, pp. 855–864.

Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. Advances in neural information processing systems 30.

Hu, B., Wu, Y., Peng, X., Sha, C., Wang, X., Fu, B., Zhao, W., 2022. Predicting change propagation between code clone instances by graph-based deep learning, in: Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension, pp. 425–436.

Hu, B., Wu, Y., Peng, X., Sun, J., Zhan, N., Wu, J., 2021. Assessing code clone harmfulness: Indicators, factors, and counter measures, in: 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE. pp. 225–236.

Hua, W., Sui, Y., Wan, Y., Liu, G., Xu, G., 2020. Fcca: Hybrid code representation for functional clone detection using attention networks. IEEE Transactions on Reliability 70, 304–318.

Inoue, R., Higo, Y., 2024. Improving accuracy of llm-based code clone detection u sing functionally equivalent methods, in: 2024 IEEE/ACIS 22nd International Conference on Software Engineering Research, Management and Applications (SERA), IEEE. pp. 24–27.

Ishihara, T., Hotta, K., Higo, Y., Igaki, H., Kusumoto, S., 2012. Inter-project functional clone detection toward building libraries: an empirical study on 13,000 projects, in: Proceedings of the 19th Working Conference on Reverse Engineering (WCRE'12).

javalang, 2020. javalang. https://github.com/c2nes/javalang.

Jia, J., Li, R., Wu, C., Feng, Y., Ma, S., Wang, L., Deng, R., 2025. Environment-adaptive representation interaction for privacy-perceptual gnns against deceptive ood attacks. IEEE Transactions on Information Forensics and Security .

Jia, J., Song, J., Yu, J., Guo, J., Jia, X., Wu, D., Yuan, Y., Cheng, G., 2026. Pagpl: Privacy-aware graph prompt learning scheme via adaptive perturbation-estimated topology recovery, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 22247–22255.

Jiang, B., Ruffolo, M., Kulkarni, A., Canini, T., AlOmar, E.A., 2026a. A systematic mapping study on the research landscape of llm-based code clone detection. Information and Software Technology , 108096.

Jiang, J., Wang, F., Shen, J., Kim, S., Kim, S., 2026b. A survey on large language models for code generation. ACM Transactions on Software Engineering and Methodology 35, 1–72.

Jiang, L., Misherghi, G., Su, Z., Glondu, S., 2007. Deckard: scalable and accurate tree-based detection of code clones, in: Proceedings of the 29th International Conference on Software Engineering (ICSE'07).

Johnson, J.H., 1994. Substring matching for clone detection and change tracking., in: Proceedings of the 1994 International Conference on Software Maintenance (ICSM'94).

Kamiya, T., Kusumoto, S., Inoue, K., 2002. Ccfinder: a multilinguistic token-based code clone detection system for large scale source code. IEEE Transactions on Software Engineering .

Kapser, C.J., Godfrey, M.W., 2006. Supporting the analysis of clones in software systems: Research articles. J. Softw. Maint. Evol. 18, 61–82.

Keivanloo, I., Rilling, J., Charland, P., 2011. Internet-scale real-time code clone search via multi-level indexing, in: Proceedings of the 18th Working Conference on Reverse Engineering (WCRE'11).

Kim, M., Sazawal, V., Notkin, D., Murphy, G.C., 2005. An empirical study of code clone genealogies, in: ESEC/FSE-13.

Komondoor, R., Horwitz, S., 2001. Using slicing to identify duplication in source code, in: Proceedings of the 2001 International Static Analysis Symposium (ISAS'01).

Koschke, R., 2012. Large-scale inter-system clone detection using suffix trees, in: Proceedings of the 16th European Conference on Software Maintenance and Reengineering (ECSME'12).

Krinke, J., 2001. Identifying similar code with program dependence graphs, in: Proceedings of the 8th Working Conference on Reverse Engineering (WCRE'01).

Li, R., Allal, L.B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., Marone, M., Akiki, C., Li, J., Chim, J., et al., 2023. Starcoder: may the source be with you! arXiv preprint arXiv:2305.06161 .

Li, Z., Huang, B., Li, Y., Mo, R., Liang, P., Liu, H., Ma, Y., 2026. Unveiling code clones in the eclipse iiot software ecosystem. Journal of Systems and Software , 112869.

Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al., 2024. Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437 .

Liu, F., Yao, Y., Guo, P., Yang, Z., Lin, X., Zhao, Z., Tong, X., Mao, K., Lu, Z., Wang, Z., et al., 2026. A systematic survey on large language models for algorithm design. ACM Computing Surveys 58, 1–32.

Liu, J., Zeng, J., Wang, X., Liang, Z., 2023. Learning graph-based code representations for source-level functional similarity detection, in: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), IEEE. pp. 345–357.

Mondal, M., Roy, B., Roy, C.K., Schneider, K.A., 2019. An empirical study on bug propagation through code cloning. Journal of Systems and Software 158, 110407.

Mou, L., Li, G., Zhang, L., Wang, T., Jin, Z., 2016. Convolutional neural networks over tree structures for programming language processing, in: Proceedings of the AAAI conference on artificial intelligence.

Moumoula, M.B., Kabore, A.K., Klein, J., Bissyande, T., 2024. Large language models for cross-language code clone detection. arXiv preprint arXiv:2408.04430 .

Nakagawa, T., Higo, Y., Kusumoto, S., 2021. Nil: large-scale detection of large-variance clones, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 830–841.

NetworkX, 2020. Software for complex networks (NetworkX). http://networkx.github.io.

Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al., 2022. Training language models to follow instructions with human feedback. Advances in neural information processing systems 35, 27730–27744.

Palumbo, E., Rizzo, G., Troncy, R., Baralis, E., Osella, M., Ferro, E., 2018. Knowledge graph embeddings with node2vec for item recommendation, in: The Semantic Web: ESWC 2018 Satellite Events: ESWC 2018 Satellite Events, Heraklion, Crete, Greece, June 3-7, 2018, Revised Selected Papers 15, Springer. pp. 117–120.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al., 2019. Pytorch: An imperative style, high-performance deep learning library. Advances

in neural information processing systems 32.

Patenaude, J.F., Merlo, E., Dagenais, M., Lague, B., 1999a. Extending software quality assessment techniques to java systems, in: Proceedings Seventh International Workshop on Program Comprehension, pp. 49–56. doi:10.1109/WPC.1999.777743.

Patenaude, J.F., Merlo, E., Dagenais, M., Laguë, B., 1999b. Extending software quality assessment techniques to java systems, in: Proceedings of the 7th International Workshop on Program Comprehension (IWPC'99).

PyTorch, 2020. Tensors and dynamic neural networks in Python with strong GPU acceleration (PyTorch). https://pytorch.org.

Ragkhitwetsagul, C., Krinke, J., 2019. Siamese: scalable and incremental code clone search via multiple code representations. Empirical Software Engineering 24, 2236–2284.

Rahman, M.K., Sujon, M.H., Azad, A., 2022. Scalable force-directed graph representation learning and visualization. Knowledge and Information Systems 64, 207–233.

Rattan, D., Bhatia, R., Singh, M., 2013. Software clone detection: A systematic review. Information and Software Technology 55, 1165–1199.

Roy, C.K., Cordy, J.R., 2007. A survey on software clone detection research. Queen's School of Computing TR .

Roy, C.K., Cordy, J.R., 2008. Nicad: accurate detection of near-miss intentional clones using flexible pretty-printing and code normalization, in: Proceedings of the 2008 International Conference on Program Comprehension (ICPC'08).

Saini, V., Farmahinifarahani, F., Lu, Y., Baldi, P., Lopes, C.V., 2018. Oreo: detection of clones in the twilight zone, in: Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE'18).

Saini, V., Farmahinifarahani, F., Sajnani, H., Lopes, C., 2021. Oreo: Scaling clone detection beyond near-miss clones, in: Code Clone Analysis: Research, Tools, and Practices. Springer, pp. 63–74.

Sajnani, H., Saini, V., Svajlenko, J., Roy, C.K., Lopes, C.V., 2016. Sourcerercc: scaling code clone detection to big code, in: Proceedings of the 38th International Conference on Software Engineering (ICSE'16).

Samoladas, I., Gousios, G., Spinellis, D., Stamelos, I., 2008. The sqo-oss quality model: measurement based open source software evaluation, in: Open Source Development, Communities and Quality: IFIP 20 th World Computer Congress, Working Group 2.3 on Open Source Software, September 7-10, 2008, Milano, Italy 4, Springer. pp. 237–248.

Shan, J., Dou, S., Wu, Y., Wu, H., Liu, Y., 2023. Gitor: Scalable code clone detection by building global sample graph, in: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 784–795.

Sheneamer, A., Kalita, J., 2016. Semantic clone detection using machine learning, in: Proceedings of the 15th International Conference on Machine Learning and Applications (ICMLA'16).

Sun, J., Bandyopadhyay, B., Bashizade, A., Liang, J., Sadayappan, P., Parthasarathy, S., 2019. Atp: Directed graph embedding with asymmetric transitivity preservation, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 265–272.

Svajlenko, J., Islam, J.F., Keivanloo, I., Roy, C.K., Mia, M.M., 2014. Towards a big data curated benchmark of inter-project code clones, in: Proceedings of the 2014 International Conference on Software Maintenance and Evolution (ICSME'14).

Svajlenko, J., Roy, C.K., 2017. Cloneworks: a fast and flexible large-scale near-miss clone detection tool., in: ICSE (Companion Volume), pp. 177–179.

Tai, K.S., Socher, R., Manning, C.D., 2015. Improved semantic representations from tree-structured long short-term memory networks. arXiv preprint arXiv:1503.00075 .

Tang, J., Qu, M., Wang, M., Zhang, M., Yan, J., Mei, Q., 2015. Line: Large-scale information network embedding, in: Proceedings of the 24th international conference on world wide web, pp. 1067–1077.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al., 2023. Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971 .

Wan, Z., Xie, C., Zeng, Y., Hu, Y., 2026. Code clone detection based on semantic images. Information and Software Technology , 108155.

Wang, M., Wang, P., Xu, Y., 2017. Ccsharp: an efficient three-phase code clone detector using modified pdgs, in: Proceedings of the 24th Asia-Pacific Software Engineering Conference (APSEC'17).

Wang, P., Svajlenko, J., Wu, Y., Xu, Y., Roy, C.K., 2018. Ccaligner: a token based large-gap clone detector, in: Proceedings of the 40th International Conference on Software Engineering (ICSE'18).

Wang, W., Deng, Z., Xue, Y., Xu, Y., 2023. Ccstokener: Fast yet accurate code clone detection with semantic token. Journal of Systems and Software 199, 111618.

Wei, H., Li, M., 2017. Supervised deep features for software functional clone detection by exploiting lexical and syntactical information in source code, in: Proceedings of the 2017 International Joint Conferences on Artificial Intelligence (IJCAI'17).

White, M., Tufano, M., Vendome, C., Poshyvanyk, D., 2016. Deep learning code fragments for code clone detection, in: Proceedings of the 31st International Conference on Automated Software Engineering (ASE'16).

Wu, M., Wang, P., Yin, K., Cheng, H., Xu, Y., Roy, C.K., 2020a. Lvmapper: A large-variance clone detector using sequencing alignment approach. IEEE access 8, 27986–27997.

Wu, Y., Zou, D., Dou, S., Yang, S., Yang, W., Cheng, F., Liang, H., Jin, H., 2020b. Scdetector: Software functional clone detection based on semantic tokens analysis, in: Proceedings of the 35th IEEE/ACM international conference on automated software engineering, pp. 821–833.

Yan, W., Xu, J., Liu, J., Yue, G., Tang, C., 2022. Bipartite graph-based discriminative feature learning for multi-view clustering, in: Proceedings of the 30th ACM International Conference on Multimedia, pp. 3403–3411.

Zakeri-Nasrabadi, M., Parsa, S., Ramezani, M., Roy, C., Ekhtiarzadeh, M., 2023. A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges. Journal of Systems and Software 204, 111796.

Zhang, J., Dong, Y., Wang, Y., Tang, J., Ding, M., 2019a. Prone: Fast and scalable network representation learning., in: IJCAI, pp. 4278–4284.

Zhang, J., Wang, X., Zhang, H., Sun, H., Wang, K., Liu, X., 2019b. A novel neural source code representation based on abstract syntax tree, in: Proceedings of the 41st International Conference on Software Engineering (ICSE'19).

Zhang, Q., Fang, C., Xie, Y., Zhang, Y., Yu, S., Sun, W., Yang, Y., Chen, Z., 2026. A survey on large language models for software engineering. Science China Information Sciences 69, 141102.

Zhao, G., Huang, J., 2018. Deepsim: deep learning code functional similarity, in: Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE'18).

Zheng, R., Dou, S., Gao, S., Hua, Y., Shen, W., Wang, B., Liu, Y., Jin, S., Liu, Q., Zhou, Y., et al., 2023. Secrets of rlhf in large language models part i: Ppo. arXiv preprint arXiv:2307.04964 .

Zou, Y., Ban, B., Xue, Y., Xu, Y., 2020. Ccgraph: a pdg-based code clone detector with approximate graph matching, in: Proceedings of the 35th IEEE/ACM international conference on automated software engineering, pp. 931–942.